

Mémoire de Projet d'Innovation

Titre du PI :

GoReview - AI Outil CLI d'analyse architecturale de code source

Soutenu le 02/06/2026

Réalisé par :

Youness HATTABI
Badr HOUSSY
Youssef NAZIH
Youssef RAHIOUI
Abdellatif MACHNAOUI

Encadré par :

Pr. Azeddine KHIAT
Pr. Fatiha AKEF

Année Universitaire : 2025-2026



Département Mathématiques et Informatique

Remerciements

Nous souhaitons exprimer notre profonde gratitude à toutes les personnes ayant contribué, de près ou de loin, à la réalisation de ce projet d'innovation.

Nous adressons tout particulièrement nos sincères remerciements à nos encadrants pédagogiques de l'École Normale Supérieure de l'Enseignement Technique de Mohammedia pour leur accompagnement, leurs conseils et leur disponibilité tout au long de ce projet. Leurs orientations et leur soutien ont joué un rôle essentiel dans l'avancement de notre travail et dans la réussite de cette expérience académique.

Nous remercions également l'ensemble de nos enseignants pour la qualité de la formation dispensée, les compétences techniques et méthodologiques acquises durant notre parcours, ainsi que leur engagement constant dans notre apprentissage.

Nos remerciements s'adressent aussi à tous les membres de notre équipe projet pour leur implication, leur sérieux et leur esprit de collaboration. Le travail collectif, les échanges d'idées et l'entraide ont largement contribué à la réussite de GoReview AI.

Nous tenons également à remercier nos amis et camarades pour leurs encouragements et leur soutien tout au long de cette période.

Enfin, nous exprimons notre reconnaissance à nos familles pour leur soutien moral, leur confiance et leurs encouragements permanents durant l'ensemble de notre parcours universitaire.

À toutes ces personnes, nous adressons nos sincères remerciements.

Résumé

Ce projet consiste en la conception et la réalisation de GoReview AI, une plateforme d'analyse architecturale de projets logiciels combinant analyse statique et intelligence artificielle. L'objectif principal est d'aider les développeurs et les responsables techniques à identifier automatiquement les problèmes de maintenabilité, de qualité logicielle et d'architecture présents dans un projet informatique. L'application permet d'analyser des projets développés principalement en Go et Python, de générer des scores de qualité, de détecter différents problèmes architecturaux et de produire des recommandations de refactorisation enrichies par des modèles d'intelligence artificielle. Le système intègre également des mécanismes d'optimisation du contexte afin de réduire la consommation de ressources lors des interactions avec les modèles LLM distants. Grâce à cette solution, les utilisateurs disposent d'un outil capable d'automatiser une partie importante des analyses architecturales, de faciliter le suivi de la qualité logicielle et d'améliorer la prise de décision durant le cycle de développement logiciel.

Abstract

This project consists of the design and implementation of GoReview AI, a software architecture analysis platform combining static analysis and artificial intelligence. The main objective is to help developers and technical leads automatically identify maintainability, software quality and architectural issues within software projects. The application is capable of analyzing projects mainly written in Go and Python, generating quality scores, detecting architectural problems and producing AI-assisted refactoring recommendations. The system also integrates context optimization mechanisms in order to reduce resource consumption during interactions with remote LLM models. Thanks to this solution, users benefit from a tool capable of automating a significant part of architectural analysis tasks, improving software quality monitoring and facilitating decision-making throughout the software development lifecycle.

Table des matières

Remerciements	iii
Résumé	iv
Abstract.....	iv
Table des matières	v
Liste des figures.....	vii
Liste des tableaux	viii
Liste des abréviations	ix
Introduction générale.....	1
Chapitre I. Contexte du projet	3
I.1 PRESENTATION DU CAHIER DES CHARGES.....	3
I.1.1 Description du contexte actuel	3
I.1.2 Solution apportée.....	4
I.1.3 Les exigences fonctionnelles.....	5
I.1.4 Exigences non fonctionnelles :.....	6
I.1.5 Comparaison avec les solutions existantes :.....	7
I.2 PLANIFICATION ET METHODOLOGIE	9
I.2.1 Méthodologie de travail.....	9
I.2.2 Etapes du projet : Diagramme de GANTT	11
I.3 CONCLUSION	13
Chapitre II. Conception.....	14
II.1 INTRODUCTION	14
II.2 CONCEPTION FONCTIONNELLE	15
II.2.1 Identification des acteurs.....	15
II.2.2 Diagramme de cas d'utilisation	16
II.2.3 Diagrammes de classe	17
II.2.3.1 Diagramme des classes principales.....	17
II.2.3.2 Architecture du moteur d'analyse	18
II.2.3.3 Structures internes d'analyse	19
II.2.3.4 Système de scoring et génération des rapports	19
II.2.4 Diagrammes de séquence :	20
II.2.4.1 Séquence d'initialisation et de sélection du mode d'analyse	20
II.2.4.2 Séquence d'analyse du projet.....	21
II.2.4.3 Séquence d'interaction avec les modèles d'intelligence artificielle.....	22
II.2.4.4 Séquence de génération des scores et des rapports	23
II.2.5 Diagramme de composants.....	24
II.2.6 Diagramme de déploiement.....	25
II.3 CONCEPTION TECHNIQUE.....	27
II.3.1 Architecture Applicative	27
II.3.1.1 Interface CLI & Orchestration	28
II.3.1.2 Modes d'Analyse	28
II.3.1.3 Détecteurs de Problèmes Architecturaux.....	29

II.3.1.4	Génération des rapports	30
II.3.1.5	Module d'intelligence artificielle.....	30
II.3.1.6	Système de scoring	31
II.4	CONCLUSION.....	31
Chapitre III.	Réalisation du projet.....	32
III.1	INTRODUCTION	32
III.2	OUTILS DE DEVELOPPEMENT	32
III.2.1	Editeur de code	32
III.2.2	Langage de programmation	33
III.2.2.1	Go	33
III.2.2.2	Comparaison entre les langages de programmation.....	33
III.2.3	Framework et bibliothèques.....	34
III.2.3.1	Cobra	34
III.2.3.2	Bibliothèques natives d'analyse AST	35
III.2.3.3	Ollama et OpenAI.....	35
III.2.3.4	Bibliothèques de génération de rapports	36
III.2.4	Détecteurs implémentés	36
III.2.5	Autre applications et services	37
III.3	IMPLEMENTATION	37
III.3.1	Menu d'aide et options CLI.....	38
III.3.2	Analyse statique et enrichissement LLM.....	38
III.3.3	Mode analyse approfondie et score final	40
III.3.4	Rapport HTML interactif.....	40
III.4	LIMITES ACTUELLES DU PROJET	42
III.5	CONCLUSION.....	42
	Conclusion générale	43
	Bibliographie	45

Liste des figures

Figure 1 - Solution apportée.....	5
Figure 2 - User stories dans l'espace de travail Notion.....	10
Figure 3 - Diagramme de GANTT	12
Figure 4 - Diagramme de cas d'utilisation.....	16
Figure 5 - Diagramme des classes principales	18
Figure 6 - Diagramme du moteur d'analyse.....	19
Figure 7 - Diagramme des structures internes d'analyse	19
Figure 8 - Diagramme du système de scoring et de reporting.....	20
Figure 9 - Séquence d'initialisation et de sélection du mode d'analyse.....	21
Figure 10 - Séquence d'analyse du projet	22
Figure 11 - Séquence d'interaction avec les modèles d'intelligence artificielle	23
Figure 12 - Séquence de génération des scores et des rapports.....	23
Figure 13 - Diagramme de composants.....	25
Figure 14 - Diagramme de déploiement.....	26
Figure 15 - Architecture applicative.....	27
Figure 16 - VS Code	32
Figure 17 - Logo Go.....	33
Figure 18 - Logo Cobra.....	34
Figure 19 - Logo Ollama.....	35
Figure 20 - Logo OpenAI.....	35
Figure 21 - Menu d'aide (./main -h)	38
Figure 22 - Résultats de l'analyse statique (./main -f .)	39
Figure 23 - Enrichissement LLM avec exemples avant/après (./main --llm .).....	39
Figure 24 - Mode analyse approfondie (./main --deep .).....	40
Figure 25 - Score de maintenabilité final.....	40
Figure 26 - Vue d'ensemble du rapport HTML (score et findings par sévérité)	41
Figure 27 - Détail d'un finding avec explication LLM et refactorisation suggérée.....	41

Liste des tableaux

Tableau 1 - Exigences Fonctionnelles.....	6
Tableau 2 - Comparatif avec les solutions existantes.....	7
Tableau 3 - Les acteurs et leurs rôles	15
Tableau 4 - Niveau de sévérité d'une erreur et sa pénalité.....	31
Tableau 5 - Comparatif entre les langages de programmation.....	33
Tableau 6 - Applications et services	37

Liste des abréviations

Abréviation	Signification
API	Application Programming Interface
AST	Abstract Syntax Tree
CI/CD	Continuous Integration / Continuous Deployment
CLI	Command Line Interface
CPU	Central Processing Unit
HTTPS	HyperText Transfer Protocol Secure
JSON	JavaScript Object Notation
LLM	Large Language Model
PDF	Portable Document Format
UML	Unified Modeling Language

Introduction générale

Dans le cadre de notre formation à École Normale Supérieure de l'Enseignement Technique de Mohammedia, les étudiants sont amenés à réaliser un projet d'innovation visant à mettre en pratique les connaissances et compétences acquises tout au long du cursus universitaire. Ce projet représente une opportunité concrète d'approfondir nos compétences techniques, de travailler en équipe sur une problématique réelle et de développer une solution logicielle répondant à des besoins actuels du domaine informatique. Il permet également de se familiariser avec les différentes étapes de conception, d'analyse et de réalisation d'un projet logiciel de grande envergure.

Dans ce contexte, nous avons travaillé sur le développement de **GoReview AI**, un outil open source d'analyse architecturale destiné aux projets logiciels. L'objectif principal de cette solution est de détecter automatiquement les problèmes de conception et les mauvaises pratiques architecturales dans le code source, appelés « architectural smells », afin d'améliorer la maintenabilité et la qualité globale des applications logicielles. Contrairement aux outils classiques d'analyse architecturale qui se concentrent principalement sur les erreurs de syntaxe ou les conventions de codage, GoReview AI vise à fournir une analyse plus avancée portant sur la structure, le couplage et l'organisation générale des systèmes logiciels.

L'idée de ce projet est née du constat que de nombreux outils actuels ne permettent pas une détection efficace des problèmes architecturaux à l'échelle d'un projet complet, ou nécessitent des infrastructures lourdes et des services cloud externes. GoReview AI propose ainsi une approche locale, rapide et extensible, capable d'analyser des projets développés dans différents langages de programmation tout en préservant la confidentialité du code source. Le système intègre également des fonctionnalités d'enrichissement par intelligence artificielle afin de proposer des recommandations de refactorisation et des suggestions de conception adaptées aux problèmes détectés.

Pour répondre à ces besoins, nous avons conçu une application en ligne de commande reposant sur des techniques d'analyse architecturale, l'utilisation des arbres syntaxiques abstraits (AST) ainsi qu'une architecture modulaire facilitant l'ajout de nouveaux détecteurs et la prise en charge de nouveaux langages. La solution permet de

générer des rapports détaillés contenant les problèmes détectés, leur niveau de gravité, ainsi que des recommandations d'amélioration exploitables par les développeurs.

Le présent document est organisé en trois chapitres. Le premier chapitre présente le contexte général du projet, la problématique étudiée ainsi qu'une vue d'ensemble de la solution proposée. Le deuxième chapitre est consacré à la conception fonctionnelle et technique de l'application, en détaillant l'architecture du système ainsi que les choix technologiques adoptés. Enfin, le troisième chapitre décrit la phase de réalisation, les outils et technologies utilisés, ainsi que l'implémentation des différentes fonctionnalités de GoReview AI.

Chapitre I. Contexte du projet

I.1 Présentation du cahier des charges

Dans cette partie, nous présenterons le cahier des charges du projet GoReview AI. Celui-ci regroupe l'ensemble des besoins et exigences permettant de définir précisément les fonctionnalités attendues ainsi que les contraintes techniques du système. Nous détaillerons notamment les objectifs du projet, les besoins fonctionnels et non fonctionnels, ainsi que les problématiques auxquelles la solution proposée tente de répondre.

I.1.1 Description du contexte actuel

Actuellement, l'évaluation de la qualité architecturale des projets logiciels repose principalement sur des outils d'analyse architecturale traditionnels, des revues de code manuelles ou des plateformes cloud nécessitant l'envoi du code source vers des services externes. Bien que ces solutions soient utiles pour détecter certaines erreurs de programmation ou violations de style, elles présentent plusieurs limites lorsqu'il s'agit d'analyser la structure globale et la maintenabilité d'un système logiciel.

Les outils classiques tels que les linters se concentrent principalement sur des problèmes locaux liés à la syntaxe, aux conventions de codage ou aux erreurs potentielles d'exécution. Ils ne permettent généralement pas d'identifier efficacement les problèmes architecturaux plus complexes tels que les God Structs, les dépendances concrètes excessives, la forte complexité cyclomatique ou encore les problèmes de couplage entre composants.

Par ailleurs, plusieurs solutions modernes basées sur l'intelligence artificielle fonctionnent exclusivement à travers des services cloud. Cette approche soulève des problématiques importantes liées à la confidentialité du code source, aux coûts d'utilisation ainsi qu'à la dépendance envers des infrastructures externes. De plus, ces outils analysent souvent uniquement des modifications ponctuelles du code plutôt que l'ensemble du projet, limitant ainsi leur capacité à détecter des problèmes architecturaux évoluant progressivement au fil du temps.

L'analyse architecturale d'un projet logiciel reste donc une tâche complexe, souvent réalisée manuellement par des développeurs expérimentés ou des architectes

logiciels. Cette approche demande un temps important, manque d'automatisation et devient difficilement applicable à grande échelle dans des projets volumineux et multi-langages.

I.1.2 Solution apportée

Afin de répondre à ces limitations, nous avons conçu GoReview AI, un outil open source d'analyse architecturale destiné à automatiser la détection des problèmes de conception au sein des projets logiciels.

La solution proposée repose sur une architecture modulaire permettant d'analyser différents langages de programmation à travers l'utilisation des arbres syntaxiques abstraits (AST) et de mécanismes d'analyse architecturale avancés. L'application fonctionne principalement sous forme d'interface en ligne de commande (CLI), offrant une solution légère, rapide et facilement intégrable dans les environnements de développement et les pipelines d'intégration continue.

L'application permet notamment :

- **D'analyser automatiquement un projet logiciel afin de détecter différents problèmes architecturaux tels que les God Structs, les dépendances concrètes, la complexité excessive ou encore les problèmes de conception liés aux types primitifs,**
- **De générer des rapports détaillés contenant les anomalies détectées, leur niveau de gravité ainsi que des recommandations de correction adaptées,**
- **D'utiliser des modèles d'intelligence artificielle pour enrichir les résultats de l'analyse et proposer des suggestions de refactorisation ainsi que des exemples d'amélioration du code,**
- **De fonctionner localement sans nécessiter l'envoi du code source vers des services externes, garantissant ainsi une meilleure confidentialité des données,**
- **De supporter plusieurs langages de programmation grâce à une architecture extensible basée sur des détecteurs indépendants,**
- **D'exporter les résultats d'analyse sous différents formats exploitables tels que JSON, HTML ou rapports textuels afin de faciliter leur consultation et leur intégration dans des workflows de développement.**

Ainsi, GoReview AI se positionne comme une solution moderne permettant d'améliorer la qualité architecturale des projets logiciels tout en réduisant le travail manuel nécessaire à l'analyse de maintenabilité. L'outil offre aux développeurs une meilleure visibilité sur les risques techniques présents dans leurs applications et facilite la prise de décisions concernant l'évolution et la refactorisation des systèmes logiciels.



Figure 1 - Solution apportée

I.1.3 Les exigences fonctionnelles

Les exigences fonctionnelles définissent l'ensemble des services et fonctionnalités attendus de l'application GoReview AI. Elles traduisent les besoins des développeurs et des équipes techniques souhaitant analyser la qualité architecturale de leurs projets logiciels de manière automatisée et structurée. L'objectif principal de cette solution est de faciliter la détection des problèmes de conception logicielle, d'améliorer la maintenabilité

des systèmes et de fournir des recommandations pertinentes permettant d’orienter les opérations de refactorisation. La solution développée vise ainsi à proposer un environnement d’analyse centralisé, extensible et automatisé capable de fonctionner sur différents projets logiciels tout en restant léger et simple d’utilisation. Les principales fonctionnalités de l’application sont présentées ci-dessous :

Tableau 1 - Exigences Fonctionnelles

Fonctionnalité	Description
Analyse de projets logiciels	Permet d’analyser automatiquement un projet logiciel afin d’identifier les problèmes architecturaux présents dans le code source.
Détection des architectural smells	Détecte différents problèmes de conception tels que les <i>God Structs</i> , les dépendances concrètes, la complexité cyclomatique élevée ou les problèmes liés aux types primitifs.
Analyse multi-langages	Supporte plusieurs langages de programmation grâce à une architecture modulaire et extensible basée sur des parseurs AST.
Génération de rapports	Produit des rapports détaillés contenant les anomalies détectées, leur niveau de gravité et des suggestions d’amélioration.
Export des résultats	Permet d’exporter les résultats d’analyse sous différents formats tels que JSON, HTML et rapports textuels.
Intégration de l’intelligence artificielle	Offre un mode d’enrichissement basé sur des modèles d’intelligence artificielle afin de générer des recommandations de refactorisation et des exemples d’amélioration du code.
Fonctionnement local	Permet une analyse locale sans obligation d’envoyer le code source vers des services cloud externes.
Architecture extensible	Facilite l’ajout de nouveaux détecteurs, nouveaux langages et nouveaux fournisseurs LLM sans modification majeure du noyau de l’application.
Intégration CI/CD	Permet l’exécution automatique des analyses dans des pipelines d’intégration continue avec définition de seuils minimaux de qualité.

I.1.4 Exigences non fonctionnelles :

En complément des fonctionnalités principales, GoReview AI doit répondre à plusieurs exigences non fonctionnelles garantissant la qualité, la robustesse et la maintenabilité de la solution. Ces exigences concernent principalement les performances, la confidentialité, l’évolutivité ainsi que la facilité d’intégration de l’application dans différents environnements de développement.

Elles se déclinent comme suit :

- **Performance** : l'analyse d'un projet logiciel doit être réalisée rapidement afin de permettre une utilisation fluide, y compris sur des projets volumineux comportant plusieurs fichiers et modules.
- **Fiabilité** : les résultats générés doivent être cohérents et reproductibles afin de garantir une détection fiable des problèmes architecturaux.
- **Confidentialité** : l'application doit privilégier une exécution locale afin d'éviter l'exposition du code source à des services externes. En mode intelligence artificielle, seules les métadonnées nécessaires à l'analyse doivent être transmises.
- **Portabilité** : l'outil doit pouvoir fonctionner sur plusieurs systèmes d'exploitation, notamment Windows et Linux, sans nécessiter de configuration complexe.
- **Facilité d'utilisation** : l'interface en ligne de commande doit être simple, claire et facilement exploitable par les développeurs et les équipes techniques.
- **Évolutivité** : l'architecture du système doit permettre l'ajout futur de nouveaux langages de programmation, de nouveaux détecteurs ainsi que de nouveaux modèles d'intelligence artificielle.
- **Modularité** : les différents composants de l'application doivent être indépendants afin de faciliter la maintenance et l'évolution du projet.
- **Respect des standards** : les analyses réalisées doivent s'appuyer sur des principes reconnus du génie logiciel et des bonnes pratiques d'architecture logicielle.

I.1.5 Comparaison avec les solutions existantes :

Avant de développer GoReview AI, il était important d'étudier les solutions déjà existantes dans le domaine de l'analyse de code et de la qualité logicielle. Cette étude comparative nous a permis d'identifier les limites des outils actuels et de définir les objectifs principaux de notre solution.

Tableau 2 - Comparatif avec les solutions existantes

Outil / Solution	Avantages	Inconvénients
Linters classiques	- Rapides et largement utilisés	- Peu adaptés à l'analyse

(<i>golangci-lint, pylint, eslint</i>)	- Détection efficace des erreurs de syntaxe et des mauvaises pratiques locales - Faciles à intégrer dans les workflows de développement	architecturale globale - N'identifient pas les problèmes de conception complexes - Analyse principalement locale au niveau des fichiers ou fonctions
SonarQube	- Analyse approfondie de la qualité logicielle - Supporte plusieurs langages - Nombreux indicateurs de qualité	- Infrastructure relativement lourde - Nécessite une configuration serveur et base de données - Complexité de déploiement pour les petits projets
Outils cloud basés IA (<i>CodeRabbit, Copilot, etc.</i>)	- Suggestions intelligentes basées sur l'IA - Assistance automatique au développement	- Dépendance au cloud - Problèmes potentiels de confidentialité du code - Analyse souvent limitée aux modifications ponctuelles
OpenSCAP / outils spécialisés sécurité	- Open source - Bonne automatisation des vérifications de sécurité	- Principalement orientés sécurité système - Peu adaptés à l'analyse architecturale des applications logicielles
GoReview AI	- Solution légère et open source - Analyse architecturale globale des projets - Fonctionnement local garantissant la confidentialité - Support multi-langages extensible - Génération de rapports détaillés - Intégration optionnelle de l'intelligence artificielle	- Nombre limité de détecteurs dans les premières versions - Certaines analyses avancées nécessitent des modèles IA - Support multi-langages encore en évolution

Grâce à cette comparaison, nous constatons que GoReview AI occupe une position intermédiaire entre les outils classiques d'analyse architecturale et les solutions d'intelligence artificielle modernes. L'application vise à offrir une solution légère, extensible et accessible permettant d'effectuer une analyse architecturale approfondie tout en conservant un fonctionnement local et une forte confidentialité des données.

I.2 Planification et méthodologie

Le succès d'un projet logiciel dépend fortement de la qualité de son organisation et de la méthodologie adoptée durant les différentes phases de développement. Une planification efficace permet non seulement de structurer le travail, mais également d'améliorer la communication entre les membres de l'équipe, d'assurer le respect des délais et de suivre l'évolution globale du projet. Dans le cadre du projet GoReview AI, la planification s'est articulée autour de deux dimensions complémentaires : l'organisation des tâches de développement et le suivi temporel du projet à travers un planning global et des outils collaboratifs.

I.2.1 Méthodologie de travail

Sur le plan méthodologique, nous avons adopté une approche de gestion de projet collaborative inspirée des méthodes agiles, tout en restant adaptée au contexte universitaire et à la taille de notre équipe composée de cinq membres. Plutôt que d'appliquer strictement un cadre méthodologique spécifique comme Scrum, nous avons privilégié une organisation flexible reposant sur les principes fondamentaux de l'agilité : collaboration continue, découpage du travail en tâches indépendantes, évolution incrémentale du projet et adaptation progressive aux besoins techniques rencontrés durant le développement.

Le projet a été structuré autour de fonctionnalités et d'objectifs définis sous forme de *User Stories*. Chaque *User Story* représentait un besoin précis du système, comme l'analyse d'un projet logiciel, la génération des rapports, l'intégration des modèles LLM ou encore l'exportation des résultats. Cette approche nous a permis de mieux répartir les responsabilités entre les membres de l'équipe tout en gardant une vision claire des priorités du projet.

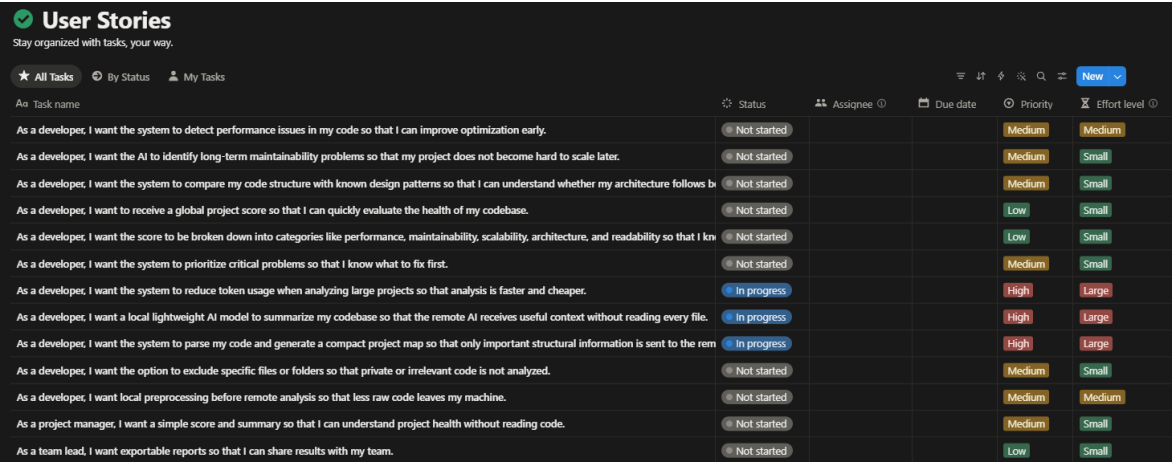
Afin d'assurer le suivi et l'organisation des tâches, nous avons utilisé la plateforme Notion comme outil central de gestion du projet. Cet outil nous a permis de :

- Créer et organiser les différentes tâches de développement,
- Centraliser les *User Stories* et les objectifs du projet,
- Regrouper les diagrammes UML et les documents de conception,
- Suivre l'avancement des différentes fonctionnalités,
- Faciliter la communication et la coordination entre les membres de l'équipe.

Le tableau de suivi du projet était organisé selon plusieurs états représentant l'évolution des tâches, notamment :

- À faire,
- En cours,
- Terminé.

Cette organisation visuelle a permis d'obtenir une vue globale et claire de l'avancement du projet tout en facilitant l'identification des priorités et des éventuels blocages techniques.



User Stories
Stay organized with tasks, your way.

★ All Tasks ⓘ By Status ⓘ My Tasks ⓘ

Task name	Status	Assignee	Due date	Priority	Effort level
As a developer, I want the system to detect performance issues in my code so that I can improve optimization early.	Not started			Medium	Medium
As a developer, I want the AI to identify long-term maintainability problems so that my project does not become hard to scale later.	Not started			Medium	Small
As a developer, I want the system to compare my code structure with known design patterns so that I can understand whether my architecture follows best practices.	Not started			Medium	Small
As a developer, I want to receive a global project score so that I can quickly evaluate the health of my codebase.	Not started			Low	Small
As a developer, I want the score to be broken down into categories like performance, maintainability, scalability, architecture, and readability so that I can identify areas for improvement.	Not started			Low	Small
As a developer, I want the system to prioritize critical problems so that I know what to fix first.	Not started			Medium	Small
As a developer, I want the system to reduce token usage when analyzing large projects so that analysis is faster and cheaper.	In progress			High	Large
As a developer, I want a local lightweight AI model to summarize my codebase so that the remote AI receives useful context without reading every file.	In progress			High	Large
As a developer, I want the system to parse my code and generate a compact project map so that only important structural information is sent to the remote AI.	In progress			High	Large
As a developer, I want the option to exclude specific files or folders so that private or irrelevant code is not analyzed.	Not started			Medium	Small
As a developer, I want local preprocessing before remote analysis so that less raw code leaves my machine.	Not started			Medium	Medium
As a project manager, I want a simple score and summary so that I can understand project health without reading code.	Not started			Medium	Small
As a team lead, I want exportable reports so that I can share results with my team.	Not started			Low	Small

Figure 2 - User stories dans l'espace de travail Notion

Parallèlement, l'utilisation de Git comme système de gestion de versions a joué un rôle essentiel dans le développement du projet. Chaque fonctionnalité développée faisait l'objet de commits réguliers accompagnés de messages descriptifs, permettant de conserver un historique précis de l'évolution du code source. Cette pratique a également facilité le travail collaboratif en limitant les conflits entre les différentes modifications réalisées par les membres de l'équipe.

L'approche adoptée nous a permis de conserver une certaine flexibilité tout au long du projet. Certaines fonctionnalités ont été réorganisées ou améliorées progressivement en fonction des résultats obtenus, des contraintes techniques rencontrées et des nouvelles idées apparues durant le développement.

En définitive, cette méthodologie nous a permis de :

- Structurer efficacement le développement du projet,
- Favoriser la collaboration et la répartition des tâches au sein de l'équipe,

- Maintenir une bonne visibilité sur l'avancement global du projet,
- Assurer un suivi clair des fonctionnalités développées,
- Faciliter l'adaptation aux contraintes et imprévus techniques,
- Garantir la continuité et la qualité du code grâce au versionnage.

Bien qu'allégée par rapport aux cadres agiles traditionnels, cette organisation s'est révélée particulièrement adaptée au contexte du projet universitaire et a permis d'assurer un développement progressif, collaboratif et cohérent de GoReview AI.

I.2.2 Etapes du projet : Diagramme de GANTT

Le diagramme de Gantt est un outil de planification de projet permettant de visualiser la répartition des tâches dans le temps. Il met en évidence :

- **Les différentes étapes du projet,**
- **Leur durée prévue,**
- **Leur enchaînement chronologique,**
- **Ainsi que les chevauchements éventuels entre certaines activités.**

Cet outil offre ainsi une vision claire de l'organisation du travail et facilite le suivi de l'avancement du projet. Le diagramme de Gantt, couvrant environ deux mois de développement (avril – mai 2026), a servi de repère visuel pour situer l'avancement global du projet et anticiper les phases clés. Bien que le calendrier ait été relativement court, il a permis de hiérarchiser les étapes, de mettre en avant les dépendances entre les tâches, et d'organiser logiquement le processus de réalisation, depuis la planification initiale jusqu'à la livraison finale. Dans notre projet, le diagramme de Gantt ci-dessus illustre l'ensemble des phases réalisées au cours de la période avril – mai 2026. Chaque tâche correspond à une étape essentielle de la mise en œuvre du système :

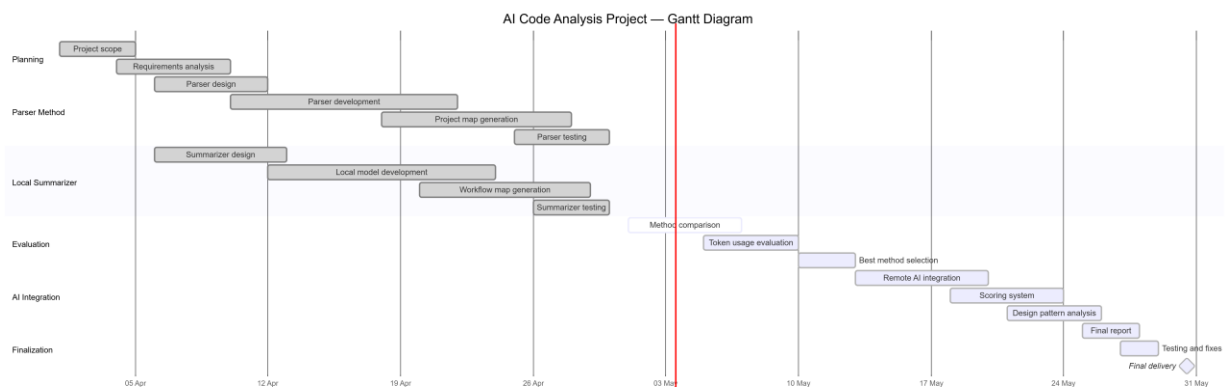


Figure 3 - Diagramme de GANTT

1. **Planification (05/04 – 10/04)** : Cette phase regroupe la définition du périmètre du projet (Project scope), l'analyse des besoins fonctionnels et techniques (Requirements analysis), ainsi que la conception de l'architecture du parser (Parser design). Elle constitue le socle sur lequel repose l'ensemble du développement.
2. **Développement de la méthode de parsing (08/04 – 29/04)** : Durant cette étape, nous avons développé le parser AST capable d'analyser les fichiers source Go et Python (Parser development), mis en place la génération de la carte de projet (Project map generation), et procédé aux tests de validation du parser (Parser testing). Ces trois tâches se sont enchaînées de manière progressive avec des chevauchements maîtrisés.
3. **Développement du résumeur local (08/04 – 29/04)** : En parallèle du parser, nous avons conçu le résumeur local (Summarizer design), développé le modèle d'inférence locale via Ollama (Local model development), généré la carte de flux de travail (Workflow map generation), et validé l'ensemble par des tests dédiés (Summarizer testing). Ce volet a constitué le cœur du mode de fonctionnement hors-ligne de GoReview AI.
4. **Évaluation et comparaison des approches (30/04 – 13/05)** : Une phase d'évaluation a été menée pour comparer les différentes méthodes de représentation du code envoyées au LLM (Method comparison) et mesurer la consommation de tokens selon chaque stratégie (Token usage evaluation). Cette analyse a permis de sélectionner la représentation optimale offrant le meilleur compromis qualité/coût.
5. **Intégration de l'IA distante (13/05 – 26/05)** : Suite à l'évaluation, nous avons sélectionné la meilleure méthode (Best method selection), intégré les providers LLM cloud (OpenAI, Anthropic) dans le pipeline (Remote AI integration), implémenté le système de scoring de maintenabilité (Scoring system), et développé l'analyse automatique des design patterns suggérés (Design pattern analysis).
6. **Finalisation et livraison (22/05 – 31/05)** : Cette dernière phase comprend la rédaction du rapport final (Final report), les tests de bout en bout et la correction

des anomalies détectées (Testing and fixes), ainsi que la livraison définitive du projet (Final delivery) prévue le 31 mai 2026.

I.3 Conclusion

Le contexte de ce projet s'inscrit dans un besoin réel lié à l'amélioration de la qualité architecturale et de la maintenabilité des projets logiciels modernes. Les solutions existantes, qu'il s'agisse des outils d'analyse architecturale traditionnels ou des plateformes cloud basées sur l'intelligence artificielle, présentent plusieurs limites en matière d'analyse architecturale globale, de confidentialité du code source et de facilité d'intégration dans des environnements de développement légers et flexibles.

À travers l'analyse des besoins et des contraintes identifiés, nous avons défini plusieurs objectifs fonctionnels et techniques pour le projet GoReview AI. L'objectif principal est de proposer une solution capable de détecter automatiquement les problèmes de conception logicielle, de générer des rapports détaillés et de fournir des recommandations pertinentes afin d'améliorer la qualité des systèmes logiciels. Le projet vise également à offrir une architecture extensible permettant la prise en charge de plusieurs langages de programmation ainsi que l'intégration de mécanismes d'intelligence artificielle pour enrichir les résultats d'analyse.

Cette phase d'étude et de cadrage constitue une étape essentielle dans le développement du projet, car elle permet de définir précisément les fonctionnalités attendues, les contraintes techniques ainsi que les orientations architecturales de la solution. Elle garantit également que l'application répondra efficacement aux besoins des développeurs et des équipes techniques tout en respectant les principes de modularité, de performance et de confidentialité.

Chapitre II. Conception

II.1 Introduction

Après avoir défini les besoins fonctionnels et non fonctionnels de GoReview AI, nous avons procédé à une phase de conception structurée intégrant les aspects fonctionnels, techniques et architecturaux de l'application. Cette étape vise à définir l'organisation globale du système, les interactions entre ses différents composants ainsi que les choix technologiques adoptés afin d'assurer la cohérence et la maintenabilité du projet.

La modélisation UML a été utilisée pour représenter de manière claire les différents composants du système, leurs relations ainsi que les interactions entre les utilisateurs et l'application. Cette approche facilite la compréhension du fonctionnement global de la solution et permet de mieux structurer les différentes phases de développement.

Sur le plan technique, GoReview AI adopte une architecture modulaire reposant sur plusieurs composants indépendants. Les différentes fonctionnalités de l'application sont organisées en modules spécialisés assurant la séparation des responsabilités entre l'analyse du code source, les détecteurs de problèmes architecturaux, le calcul des scores, la génération des rapports et l'intégration des modèles d'intelligence artificielle. Cette organisation améliore la réutilisabilité du code et simplifie les opérations de maintenance et d'évolution du système.

L'application repose principalement sur des mécanismes d'analyse architecturale utilisant les arbres syntaxiques abstraits (AST) afin de parcourir et analyser le code source des projets étudiés. Pour certains langages, des technologies telles que regex sont également utilisées afin de fournir une prise en charge multi-langages flexible et extensible.

Contrairement aux plateformes d'analyse cloud nécessitant l'envoi du code source vers des serveurs externes, GoReview AI privilégie une exécution locale garantissant la confidentialité des données analysées. Les différents composants de l'application communiquent directement entre eux à travers des structures de données internes, ce qui réduit la complexité du système et améliore ses performances.

Le système propose également plusieurs modes de génération de rapports et d'exportation des résultats d'analyse sous différents formats tels que JSON, HTML ou rapports textuels, facilitant ainsi l'intégration avec les environnements de développement et les pipelines d'intégration continue.

Cette conception technique permet d'assurer la modularité, l'évolutivité et la robustesse de l'application tout en répondant efficacement aux besoins liés à l'automatisation de l'analyse architecturale des projets logiciels.

II.2 Conception Fonctionnelle

II.2.1 Identification des acteurs

À travers l'identification des acteurs, nous définissons les différents utilisateurs et systèmes susceptibles d'interagir avec l'application GoReview AI. Dans le cadre de ce projet, le système a été conçu pour assister les développeurs et les responsables techniques dans l'analyse de la qualité architecturale des projets logiciels grâce à une combinaison d'analyse statique et d'intelligence artificielle. L'application permet aux développeurs de connecter leurs projets, de lancer différents types d'analyses, de consulter les scores de qualité générés ainsi que d'exporter les rapports produits par le système. Les responsables techniques (*Team Leads*) peuvent également suivre l'évolution de la qualité d'un projet à travers l'historique des scores et les rapports générés. Par ailleurs, l'application interagit avec plusieurs systèmes d'intelligence artificielle. Les modèles locaux sont utilisés pour effectuer certaines analyses avancées tout en conservant les données sur la machine de l'utilisateur, tandis que les modèles distants permettent d'enrichir les analyses grâce à des capacités de raisonnement plus avancées et à la génération de recommandations de refactorisation.

Tableau 3 - Les acteurs et leurs rôles

Acteur	Rôle
Développeur	- Connecter ou importer un projet logiciel - Lancer les analyses de qualité du code - Sélectionner le mode d'analyse - Générer des rapports d'analyse

	- Consulter les recommandations générées - Comparer différentes méthodes d'analyse
Team Lead	- Consulter les rapports générés - Suivre l'évolution du score qualité des projets - Analyser l'historique des évaluations - Superviser la qualité architecturale globale du projet
Modèle IA local	- Réaliser des analyses locales avancées - Générer des résumés du projet - Construire des représentations internes du workflow et de l'architecture logicielle
Modèle IA distant	- Fournir des analyses enrichies basées sur des LLM - Générer des recommandations de refactorisation - Comparer le projet avec des design patterns connus - Identifier des problèmes architecturaux complexes à long terme

II.2.2 Diagramme de cas d'utilisation

Une fois les acteurs du système identifiés, il est nécessaire de définir les principales interactions possibles avec l'application. Le diagramme de cas d'utilisation permet de représenter les différentes fonctionnalités offertes par GoReview AI ainsi que les relations entre les acteurs et les services proposés par le système.

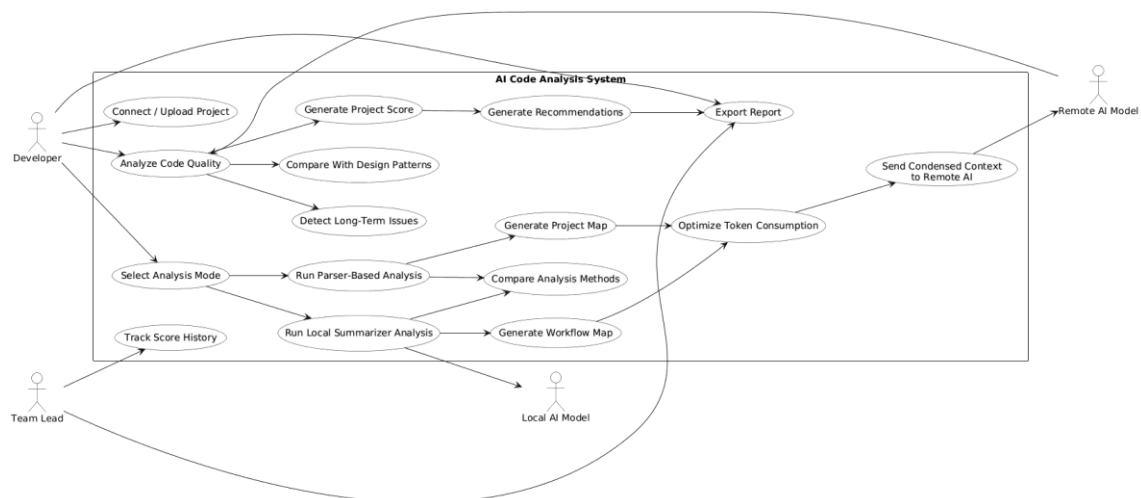


Figure 4 - Diagramme de cas d'utilisation

Dans ce projet, les cas d'utilisation principaux concernent l'analyse de la qualité logicielle, la génération des scores de maintenabilité, la production de recommandations intelligentes ainsi que l'exportation des rapports d'analyse. Le système permet également de sélectionner différents modes d'analyse, d'effectuer des comparaisons entre plusieurs méthodes d'évaluation et d'utiliser des modèles d'intelligence artificielle locaux ou distants afin d'enrichir les résultats produits. L'application intègre également des mécanismes avancés d'optimisation de contexte permettant de réduire la consommation de tokens lors des interactions avec les modèles LLM distants.

II.2.3 Diagrammes de classe

Afin de mieux représenter l'architecture interne de GoReview AI, plusieurs diagrammes de classes ont été réalisés. Chaque diagramme met en évidence une partie spécifique du système afin d'améliorer la lisibilité et de faciliter la compréhension des interactions entre les différents composants. Cette séparation permet également de mieux structurer l'analyse technique du projet en isolant les responsabilités principales de chaque module. Les diagrammes ont été organisés selon plusieurs axes fonctionnels : les entités principales manipulées par le système, le moteur d'analyse architecturale, les structures internes utilisées durant les traitements ainsi que le système de scoring et de génération des rapports. Cette approche modulaire reflète directement l'architecture réelle de l'application et facilite son évolution future.

II.2.3.1 Diagramme des classes principales

Le diagramme des classes principales représente les entités fondamentales manipulées par GoReview AI durant le cycle d'analyse d'un projet logiciel. Il permet d'illustrer les objets centraux du système ainsi que les relations qui les unissent. Dans cette partie, nous retrouvons principalement les classes représentant les projets analysés, les fichiers source, les sessions d'analyse, les scores de qualité ainsi que les rapports générés. Ces classes constituent le noyau métier de l'application et servent de point d'entrée aux différents traitements réalisés par le moteur d'analyse. Ce diagramme permet également de mettre en évidence le déroulement global du processus d'analyse : un projet est chargé par le système, analysé au cours d'une session dédiée, puis les résultats sont transformés en scores et en rapports exploitables par les utilisateurs.

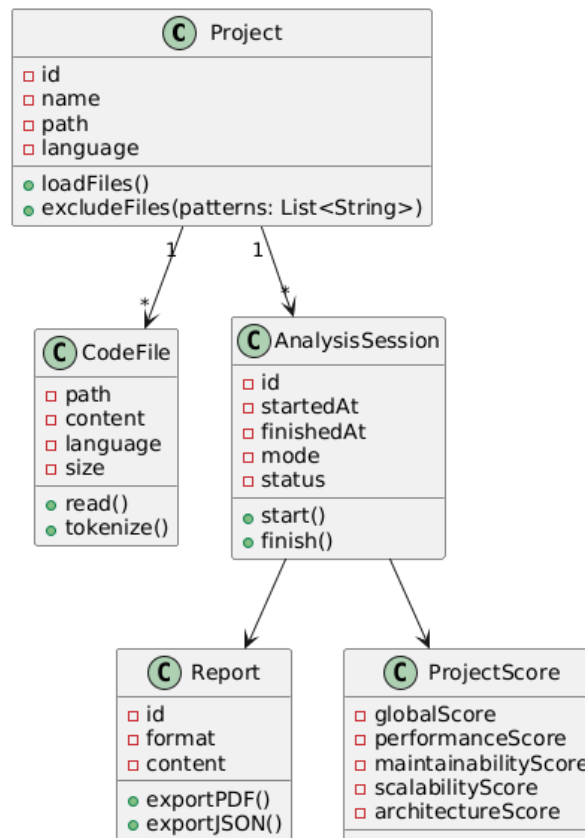


Figure 5 - Diagramme des classes principales

II.2.3.2 Architecture du moteur d'analyse

Le moteur d'analyse représente le composant principal de GoReview AI. Il regroupe l'ensemble des modules responsables de l'analyse du code source, de la génération des représentations internes du projet et de l'interaction avec les modèles d'intelligence artificielle. Cette architecture repose sur une approche hybride combinant analyse statique classique et enrichissement par intelligence artificielle. Dans un premier temps, les parseurs analysent les fichiers source afin de construire une représentation structurée du projet. Ensuite, des modules spécialisés génèrent des résumés techniques, des cartes de dépendances et des représentations simplifiées du workflow applicatif. Afin d'optimiser les interactions avec les modèles LLM distants, un composant dédié réduit et condense les informations importantes avant leur transmission. Cette approche permet de limiter la consommation de tokens tout en conservant les éléments nécessaires à une analyse pertinente.

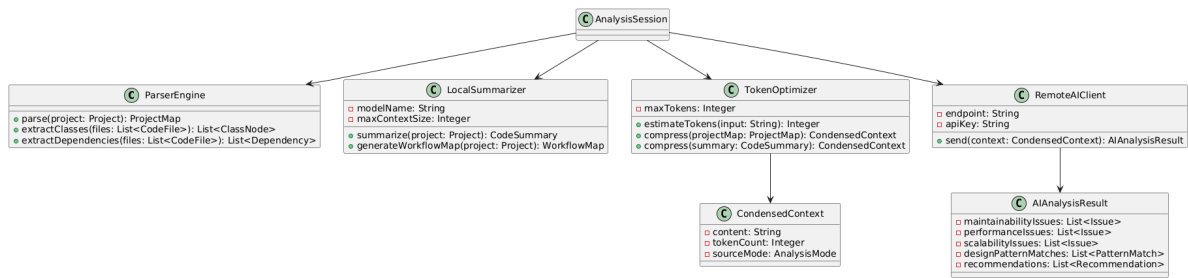


Figure 6 - Diagramme du moteur d'analyse

II.2.3.3 Structures internes d'analyse

Le fonctionnement interne de GoReview AI repose sur plusieurs structures de données intermédiaires utilisées pour représenter les informations extraites durant l'analyse du projet. Ces structures permettent notamment de modéliser l'architecture logicielle, les relations entre les composants, les flux de travail, les problèmes détectés ainsi que les recommandations générées par les modèles d'intelligence artificielle. Parmi les principales structures utilisées, nous retrouvons les représentations des modules du projet (*ProjectMap*), les cartes de workflow (*WorkflowMap*), les résumés de code (*CodeSummary*) ainsi que les objets représentant les anomalies et recommandations détectées. L'utilisation de ces structures intermédiaires facilite la séparation des responsabilités entre les différents composants du système et améliore la modularité globale de l'application. Elle permet également de rendre le pipeline d'analyse plus flexible et plus facilement extensible pour les futures évolutions du projet.

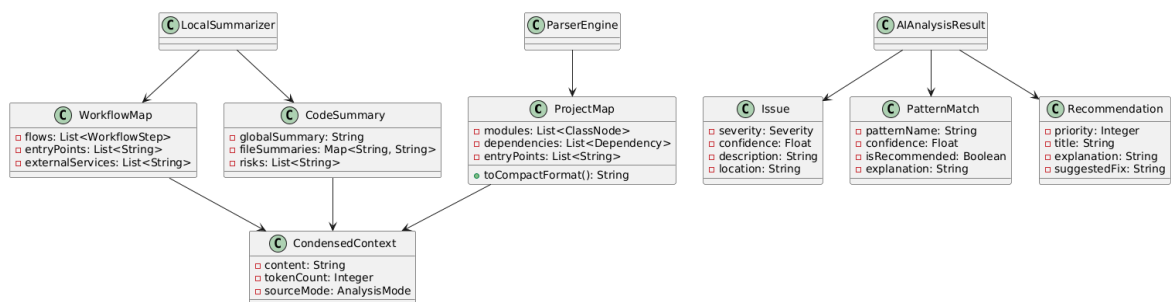


Figure 7 - Diagramme des structures internes d'analyse

II.2.3.4 Système de scoring et génération des rapports

Une fois l'analyse terminée, les résultats détectés sont traités afin de produire une évaluation globale de la qualité architecturale du projet. Cette étape est assurée par le système de scoring et de génération des rapports. Le moteur de scoring calcule différents indicateurs de qualité tels que la maintenabilité, la complexité, la scalabilité ou encore la

qualité architecturale globale du projet. Chaque problème détecté influence le score final selon son niveau de gravité et son impact potentiel sur l'évolution du système. Les résultats obtenus sont ensuite transmis au module de reporting chargé de produire des rapports détaillés exploitables par les développeurs et les responsables techniques. Ces rapports peuvent être exportés sous plusieurs formats afin de faciliter leur intégration dans des environnements CI/CD ou leur consultation manuelle.

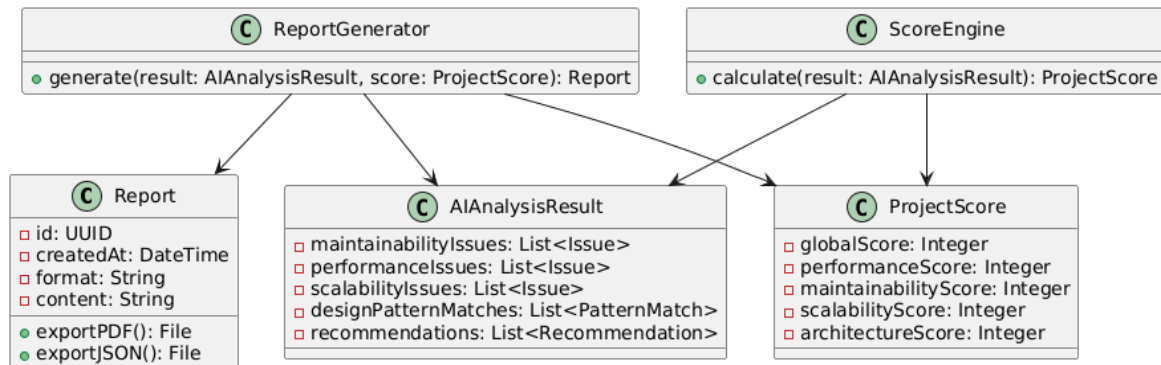


Figure 8 - Diagramme du système de scoring et de reporting

II.2.4 Diagrammes de séquence :

Les diagrammes de séquence permettent de représenter dynamiquement les échanges entre les différents composants du système au cours de l'exécution d'un scénario précis. Contrairement aux diagrammes de classes qui décrivent la structure statique de l'application, les diagrammes de séquence mettent en évidence l'ordre chronologique des interactions ainsi que le déroulement interne des traitements. Dans le cadre de GoReview AI, plusieurs diagrammes de séquence ont été réalisés afin de mieux représenter les différentes phases du pipeline d'analyse architecturale. Afin d'améliorer la lisibilité et de faciliter l'explication du fonctionnement du système, les interactions ont été divisées en plusieurs diagrammes spécialisés couvrant chacune une partie spécifique du processus d'analyse.

II.2.4.1 Séquence d'initialisation et de sélection du mode d'analyse

Ce premier diagramme illustre les premières interactions entre l'utilisateur et l'application lors du lancement d'une analyse. Le développeur sélectionne un projet logiciel à analyser, puis le système effectue une phase de scan afin d'identifier automatiquement les fichiers source, le langage utilisé ainsi que certaines métadonnées du projet. Une fois cette phase terminée, l'utilisateur choisit le mode d'analyse souhaité. Cette

sélection détermine le pipeline de traitement qui sera exécuté par le système durant les étapes suivantes.

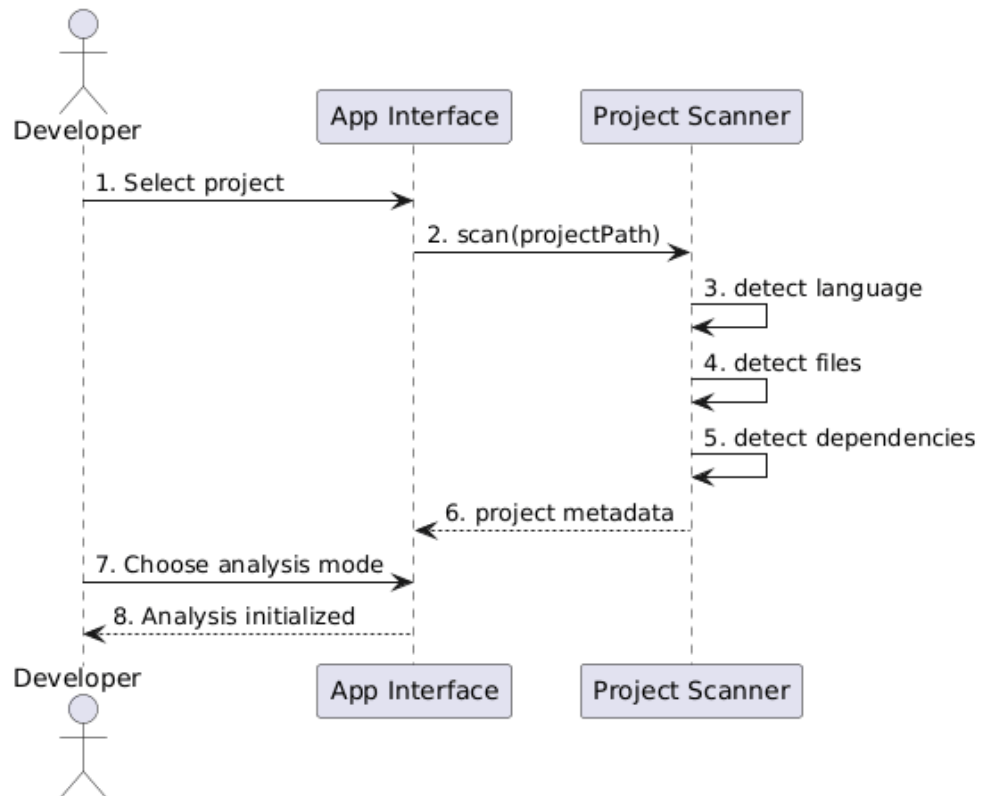


Figure 9 - Séquence d'initialisation et de sélection du mode d'analyse

II.2.4.2 Séquence d'analyse du projet

Ce diagramme représente le fonctionnement du moteur d'analyse selon les différents modes disponibles dans GoReview AI. Le système peut utiliser une analyse basée sur les parseurs, une analyse par résumé local ou encore une approche hybride combinant les deux méthodes. Dans le mode basé sur les parseurs, le système construit des représentations AST afin d'extraire les modules, classes et dépendances du projet. Dans le mode de résumé local, le système génère des représentations simplifiées du code et du workflow applicatif. Enfin, le mode hybride fusionne les résultats des deux approches afin d'obtenir une analyse plus complète.

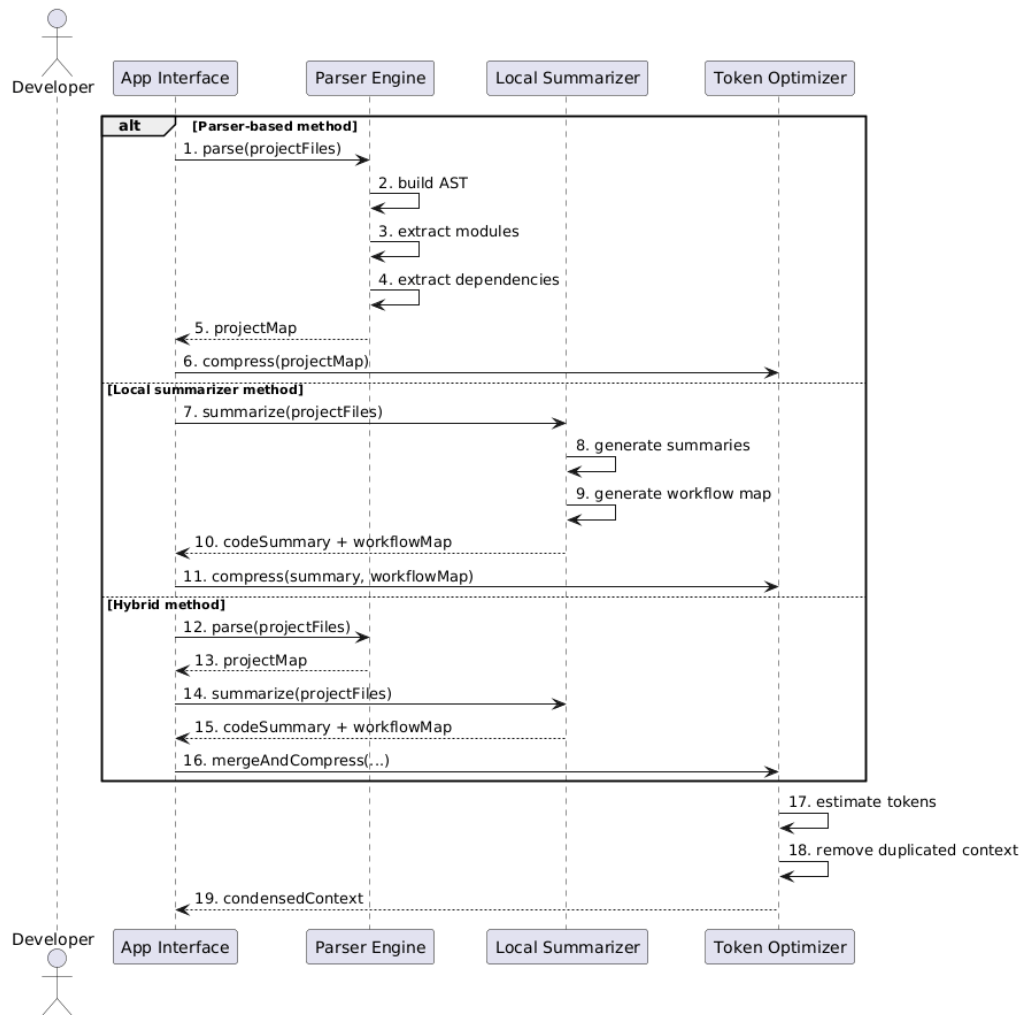


Figure 10 - Séquence d'analyse du projet

II.2.4.3 Séquence d'interaction avec les modèles d'intelligence artificielle

Cette séquence décrit les interactions entre l'application et les modèles d'intelligence artificielle distants utilisés pour enrichir les analyses architecturales. Une fois les données du projet condensées et optimisées, le système transmet le contexte généré au client d'intelligence artificielle qui communique ensuite avec le modèle LLM distant. Le modèle effectue alors plusieurs opérations d'analyse avancée, notamment la détection de problèmes de maintenabilité, l'identification de risques de performance, la comparaison avec des design patterns connus ainsi que la génération de recommandations de refactorisation. Les résultats produits sont ensuite renvoyés à l'application afin d'être traités et affichés à l'utilisateur.

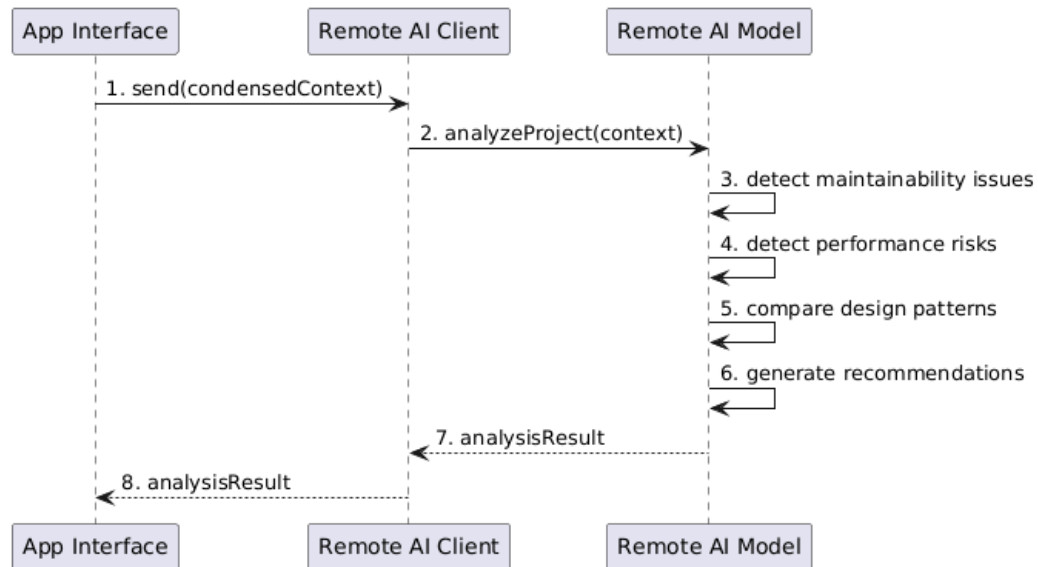


Figure 11 - Séquence d'interaction avec les modèles d'intelligence artificielle

II.2.4.4 Séquence de génération des scores et des rapports

Après la phase d'analyse, les résultats obtenus sont utilisés pour calculer les différents scores de qualité du projet. Cette étape est réalisée par le moteur de scoring qui attribue une évaluation globale en fonction des problèmes détectés et de leur niveau de gravité. Les résultats sont ensuite transmis au module de génération des rapports afin de produire un document structuré contenant les scores, les anomalies détectées ainsi que les recommandations proposées par le système. Enfin, le rapport final ainsi que les résultats synthétiques sont affichés à l'utilisateur.

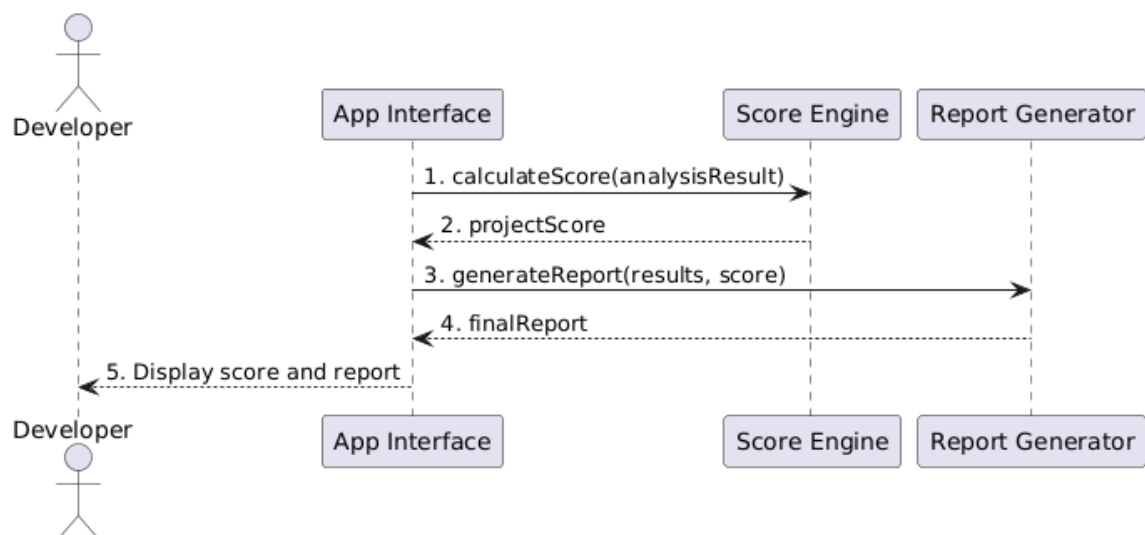


Figure 12 - Séquence de génération des scores et des rapports

II.2.5 Diagramme de composants

Le diagramme de composants permet de représenter l'architecture logicielle globale de l'application en mettant en évidence les principaux modules fonctionnels ainsi que les interactions entre eux. Contrairement aux diagrammes de classes qui se concentrent sur les structures internes du code, le diagramme de composants offre une vision plus abstraite du système en identifiant les grandes briques logicielles qui composent l'application. Dans le cadre de GoReview AI, ce diagramme illustre l'organisation générale du pipeline d'analyse architecturale ainsi que la séparation des responsabilités entre les différents modules du système. L'architecture est principalement divisée en trois parties : la couche locale exécutée sur la machine du développeur, les services d'intelligence artificielle distants ainsi que la couche de génération et d'exportation des résultats.

La première partie regroupe les composants locaux responsables du scan du projet, de l'analyse du code source, de la génération des représentations internes ainsi que de l'optimisation du contexte avant son envoi vers les modèles LLM distants. Cette approche permet de limiter les données réellement transmises aux services externes tout en réduisant la consommation de tokens.

La seconde partie représente les services d'analyse avancée basés sur l'intelligence artificielle. Ces composants sont responsables de l'identification des problèmes architecturaux, de la comparaison avec des design patterns connus, de l'analyse de la qualité logicielle ainsi que de la génération des recommandations.

Enfin, la dernière partie du diagramme correspond à la couche de restitution des résultats. Celle-ci assure la génération des rapports, l'affichage des résultats dans le tableau de bord ainsi que l'exportation des analyses sous différents formats exploitables.

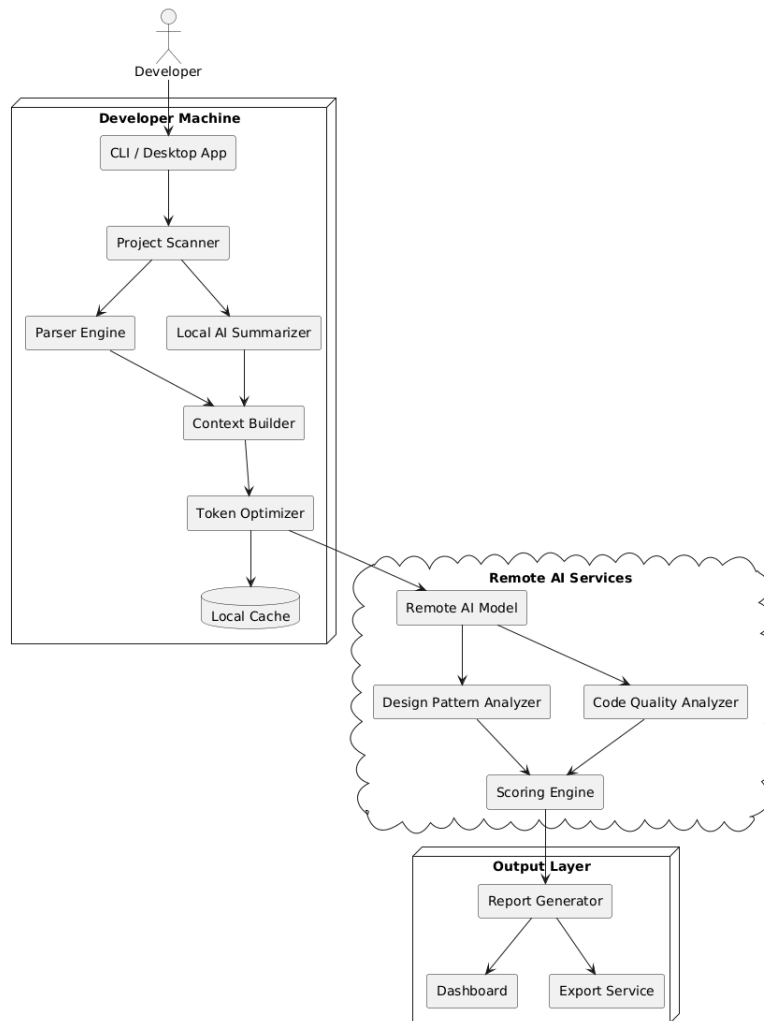


Figure 13 - Diagramme de composants

II.2.6 Diagramme de déploiement

Le diagramme de déploiement permet de représenter l'organisation physique des différents composants du système ainsi que leur répartition sur les environnements d'exécution. Il met en évidence les échanges réseau, les dépendances matérielles et les différents nœuds impliqués dans le fonctionnement global de l'application. Dans le cadre de GoReview AI, l'architecture de déploiement repose sur une approche hybride combinant des traitements locaux et des services distants d'intelligence artificielle. Une grande partie des opérations d'analyse est exécutée directement sur la machine du développeur afin de réduire les échanges réseau et de préserver la confidentialité du code source analysé.

L'application locale contient les composants responsables du scan des projets, de la génération des représentations internes, de l'analyse syntaxique ainsi que de l'optimisation

du contexte transmis aux modèles distants. Les données intermédiaires et certains résultats temporaires peuvent également être stockés dans un cache local afin d'améliorer les performances.

Les traitements avancés basés sur les modèles LLM sont quant à eux exécutés sur une plateforme distante accessible via des API sécurisées. Cette plateforme héberge plusieurs services spécialisés dans l'analyse architecturale, la détection des problèmes de qualité logicielle ainsi que la génération de recommandations intelligentes.

Les résultats finaux sont ensuite transmis à la couche de restitution qui permet d'afficher les scores, les tableaux de bord et les rapports exportables aux formats PDF ou JSON.

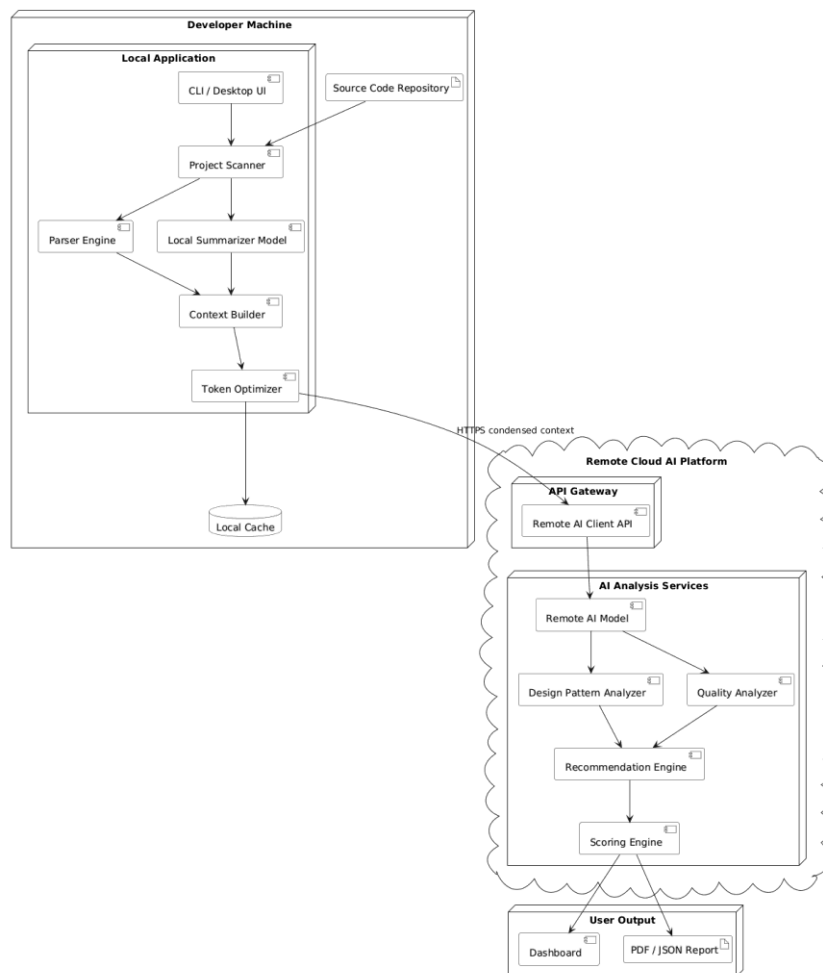


Figure 14 - Diagramme de déploiement

II.3 Conception Technique

II.3.1 Architecture Applicative

L'architecture applicative de GoReview AI repose sur une structure modulaire permettant de séparer clairement les différentes responsabilités du système. Chaque module possède un rôle spécifique dans le processus d'analyse et peut évoluer indépendamment des autres composants. Cette organisation facilite la maintenance du projet, l'ajout de nouvelles fonctionnalités ainsi que l'intégration future de nouveaux langages de programmation et de nouveaux détecteurs. :

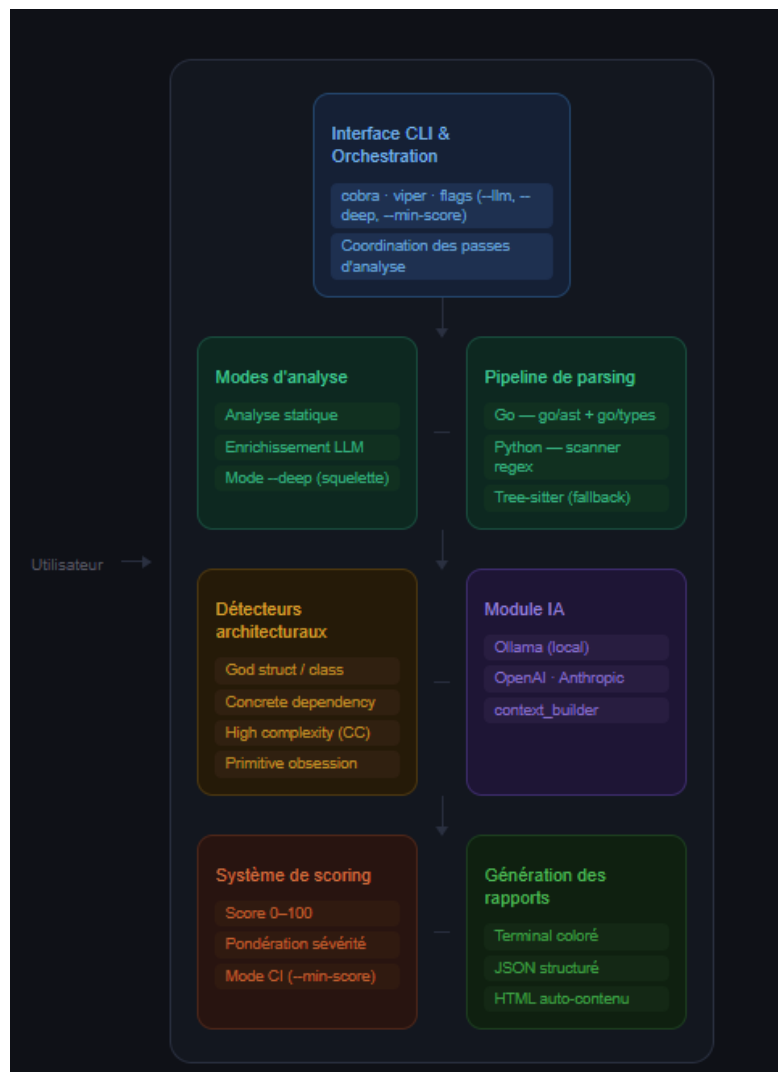


Figure 15 - Architecture applicative

Le fonctionnement interne de GoReview AI suit un pipeline d'analyse structuré composé de plusieurs étapes successives :

- **Parcours récursif des fichiers source,**

- **Construction des AST,**
- **Exécution des détecteurs statiques,**
- **Calcul du score de maintenabilité,**
- **Enrichissement optionnel par LLM,**
- **Génération et exportation des rapports.**

Cette organisation améliore la modularité du système et facilite l'ajout futur de nouveaux détecteurs ou nouveaux formats d'analyse.

II.3.1.1 Interface CLI & Orchestration

L'interaction avec l'application se fait principalement à travers une interface en ligne de commande développée en Go à l'aide du framework Cobra.

Cette interface est responsable :

- **De la réception des commandes et des paramètres fournis par l'utilisateur,**
- **Du lancement des processus d'analyse,**
- **De la configuration des options d'exécution (activation du mode LLM, export des rapports, définition des seuils CI, etc.),**
- **De l'affichage des résultats et des statistiques générées.**

Le module principal agit également comme orchestrateur du système en coordonnant les différents composants internes durant l'analyse d'un projet logiciel.

II.3.1.2 Modes d'Analyse

GoReview AI propose plusieurs modes d'analyse permettant d'adapter le niveau de profondeur et d'intelligence de l'évaluation architecturale selon les besoins de l'utilisateur.

Analyse statique

Le mode d'analyse statique constitue le fonctionnement de base de l'application. Dans ce mode, le système parcourt récursivement les fichiers source des projets Go et Python afin de construire des représentations AST (Abstract Syntax Trees) puis exécute différents détecteurs de problèmes architecturaux. Ce mode fonctionne entièrement en local, sans connexion réseau ni dépendance à des services externes.

Mode enrichi par LLM

Le mode `--llm` ajoute une couche d'intelligence artificielle au-dessus des résultats de l'analyse statique. Pour chaque problème détecté, le système génère :

- Une explication textuelle du problème,
- Une estimation de son impact sur la maintenabilité,
- Des recommandations de refactorisation,
- Des exemples d'amélioration du code.

Analyse approfondie

Le mode `--deep` permet une analyse architecturale plus avancée à l'échelle des packages et des modules du projet. Dans ce mode, GoReview AI transmet aux modèles LLM une version simplifiée de la structure du projet contenant uniquement :

- Les signatures des fonctions,
- Les structures et classes,
- Les imports et dépendances,
- Les relations entre composants.

Cette approche permet de détecter des problèmes complexes difficilement identifiables par les détecteurs statiques traditionnels, tels que :

- Les violations de couches (Layer Violations),
- Les dépendances implicites,
- Les problèmes de couplage inter-modules,
- Les modèles de conception manquants.

II.3.1.3 Détecteurs de Problèmes Architecturaux

Les détecteurs représentent des modules spécialisés chargés d'identifier des problèmes architecturaux précis au sein du code source. Chaque détecteur implémente une logique indépendante permettant d'analyser certaines métriques ou structures spécifiques du projet. Parmi les problèmes détectés par le système, nous pouvons citer :

- Les *God Structs* ou classes surchargées,
- Les dépendances concrètes excessives,

- **La complexité cyclomatique élevée,**
- **Les problèmes liés aux types primitifs (*Primitive Obsession*),**
- **Les problèmes de couplage et de maintenabilité.**

L'architecture des détecteurs est volontairement modulaire afin de faciliter l'ajout futur de nouvelles règles d'analyse sans modifier le noyau principal du système.

II.3.1.4 Génération des rapports

Une fois l'analyse terminée, les résultats détectés sont centralisés puis transformés sous différents formats de sortie.

Le module de reporting permet :

- **L'affichage des résultats directement dans le terminal,**
- **La génération de rapports structurés au format JSON,**
- **L'exportation de rapports HTML détaillés,**
- **La production de résumés exploitables dans des pipelines CI/CD.**

Chaque rapport contient les anomalies détectées, leur niveau de gravité, leur emplacement dans le code source ainsi que des suggestions d'amélioration.

II.3.1.5 Module d'intelligence artificielle

GoReview AI intègre également un module optionnel d'enrichissement basé sur des modèles d'intelligence artificielle (LLM – Large Language Models). Ce composant permet :

- **D'analyser les résultats générés par les détecteurs,**
- **De produire des recommandations de refactorisation,**
- **De suggérer des design patterns adaptés,**
- **De générer des exemples de restructuration du code.**

Le système supporte plusieurs fournisseurs LLM tels que Ollama, OpenAI et Anthropic grâce à une architecture basée sur des interfaces abstraites permettant de remplacer facilement le backend utilisé. Afin de préserver la confidentialité des projets analysés, seules les métadonnées nécessaires à l'analyse sont transmises aux modèles d'intelligence artificielle, et non le code source complet.

II.3.1.6 Système de scoring

GoReview AI intègre un système de scoring permettant d'évaluer automatiquement le niveau global de maintenabilité d'un projet logiciel. Le score est calculé sur une échelle de 0 à 100. Chaque problème architectural détecté entraîne une pénalité selon son niveau de gravité :

Tableau 4 - Niveau de sévérité d'une erreur et sa pénalité

Niveau	Pénalité
Critique	-15
Moyen	-7
Faible	-3

Le score final permet aux développeurs et aux équipes CI/CD d'obtenir une mesure synthétique de la qualité architecturale du projet. Le système peut également être utilisé dans des pipelines d'intégration continue grâce au paramètre `--min-score`, permettant de bloquer automatiquement certaines validations lorsque le score descend sous un seuil minimal défini.

II.4 Conclusion

La phase de conception a joué un rôle essentiel dans le développement de GoReview AI puisqu'elle a permis de définir clairement l'organisation technique et fonctionnelle du système. Grâce à une architecture modulaire et extensible, nous avons pu structurer l'application autour de plusieurs composants indépendants responsables de l'analyse architecturale, de la détection des problèmes architecturaux, de la génération des rapports et de l'intégration de l'intelligence artificielle. Cette étape a permis d'anticiper les besoins futurs du projet, notamment la prise en charge de nouveaux langages de programmation, l'ajout de nouveaux détecteurs ainsi que l'intégration de fonctionnalités avancées liées à l'automatisation des analyses logicielles. La conception technique a également permis de garantir une bonne séparation des responsabilités entre les différents modules du système, améliorant ainsi la maintenabilité, la réutilisabilité et l'évolutivité de l'application. Elle constitue donc une base solide pour les phases de réalisation et d'évolution future du projet.

Chapitre III. Réalisation du projet

III.1 Introduction

Après avoir présenté les aspects liés à la conception fonctionnelle et technique de GoReview AI, ce chapitre est consacré à la phase de réalisation du projet. Nous y présenterons les différents outils, technologies et bibliothèques utilisés durant le développement de l'application, ainsi que leur rôle dans l'implémentation de la solution.

Cette partie permettra également de décrire les choix techniques adoptés lors du développement, les différentes fonctionnalités réalisées ainsi que les mécanismes mis en place pour l'analyse des projets logiciels, la détection des problèmes architecturaux et la génération des rapports d'analyse.

III.2 Outils de développement

III.2.1 Editeur de code

Visual Studio Code offre un ensemble riche de fonctionnalités telles que la coloration syntaxique contextuelle, la détection d'erreurs et la complétion intelligente du code. Il dispose également d'un écosystème de plugins très fourni, facilitant considérablement le processus de développement. [1]



Figure 16 - VS Code

Pour le développement de GoReview AI, nous avons choisi Visual Studio Code comme environnement de développement principal. Ce choix s'explique par sa légèreté, sa flexibilité ainsi que son excellente intégration avec les outils de développement modernes. L'éditeur propose une prise en charge complète du langage Go grâce à plusieurs extensions facilitant le débogage, la navigation dans le code, l'analyse architecturale ainsi que l'intégration avec Git. Par ailleurs, l'intégration native du terminal et des outils de gestion

de versions a permis d'améliorer la productivité durant le développement et de simplifier l'exécution et les tests de l'application.

III.2.2 Langage de programmation

III.2.2.1 Go

Go est un langage de programmation compilé développé par Google. Il est conçu pour être simple, performant et adapté au développement d'applications concurrentes, distribuées et fortement modulaires. Le langage fournit également une bibliothèque standard riche facilitant la création d'outils système, de services réseau et d'applications CLI. [2]



Figure 17 - Logo Go

Nous avons choisi Go comme langage principal pour le développement de GoReview AI en raison de ses excellentes performances, de sa simplicité syntaxique et de sa forte compatibilité avec les outils d'analyse architecturale. Le langage fournit également des bibliothèques natives puissantes telles que `go/ast`, `go/parser` et `go/types`, essentielles pour l'analyse du code source et la construction des arbres syntaxiques abstraits (AST). Go facilite également la création d'applications multiplateformes légères et facilement distribuables sous forme de binaire unique, ce qui correspond parfaitement aux besoins du projet.

III.2.2.2 Comparaison entre les langages de programmation

Tableau 5 - Comparatif entre les langages de programmation

Langage	Avantages principaux	Inconvénients	Pertinence pour le projet
Go	Performances élevées, compilation rapide, excellente gestion de la concurrence,	Écosystème GUI limité par rapport à d'autres langages	Très adapté pour les outils CLI et l'analyse de code

	bibliothèque standard riche pour l'analyse architecturale		
Python	Simplicité, grand nombre de bibliothèques, développement rapide	Performances plus faibles, dépendance à l'interpréteur	Utilisé uniquement pour certains parseurs et analyses secondaires
Java	Robuste, multiplateforme, vaste écosystème	Syntaxe plus lourde, consommation mémoire plus importante	Adapté aux grandes applications mais moins léger pour un outil CLI
C++	Très performant, contrôle bas niveau	Complexité élevée, gestion mémoire plus difficile	Peu adapté au rythme de développement du projet
Rust	Très performant et sécurisé	Courbe d'apprentissage importante	Intéressant pour des outils systèmes mais plus complexe à développer rapidement

III.2.3 Framework et bibliothèques

III.2.3.1 Cobra

Cobra est une bibliothèque Go permettant de créer des applications en ligne de commande modernes et structurées. Elle fournit un système de gestion des commandes, des sous-commandes et des paramètres, largement utilisé dans l'écosystème Go. [3]



Figure 18 - Logo Cobra

Nous avons choisi Cobra pour développer l'interface CLI de GoReview AI car elle facilite l'organisation des commandes, la gestion des options et la structuration globale de l'application. Elle permet également d'améliorer l'expérience utilisateur grâce à une gestion claire des arguments et de l'aide intégrée.

III.2.3.2 Bibliothèques natives d'analyse AST

Les bibliothèques `go/ast`, `go/parser` et `go/types` font partie de la bibliothèque standard du langage Go. Elles permettent de parser le code source, de construire des arbres syntaxiques abstraits et d'effectuer des analyses de types.

Nous avons utilisé ces bibliothèques comme base principale du moteur d'analyse statique de GoReview AI. Elles permettent de parcourir le code source de manière structurée afin d'identifier automatiquement différents problèmes architecturaux et métriques de complexité. Leur intégration native dans Go garantit également de bonnes performances ainsi qu'une excellente compatibilité avec les projets analysés.

III.2.3.3 Ollama et OpenAI

Ollama est une plateforme permettant l'exécution locale de modèles d'intelligence artificielle générative. [4]



Figure 19 - Logo Ollama

OpenAI fournit quant à lui des modèles LLM accessibles via API et capables de produire des analyses et recommandations avancées. [5]



Figure 20 - Logo OpenAI

Ces technologies ont été intégrées dans GoReview AI afin d'ajouter un mécanisme d'enrichissement intelligent des résultats d'analyse. Les modèles LLM sont utilisés pour :

- Générer des suggestions de refactorisation,
- Recommander des design patterns adaptés,
- Produire des explications détaillées des problèmes détectés.

L'utilisation d'Ollama permet également d'exécuter certains modèles localement afin de préserver la confidentialité du code source analysé.

III.2.3.4 Bibliothèques de génération de rapports

Le projet intègre également plusieurs modules internes responsables de la génération des rapports d'analyse aux formats texte, JSON et HTML.

Ces composants permettent :

- D'afficher les résultats directement dans le terminal,
- De produire des rapports structurés exploitables dans des pipelines CI/CD,
- De faciliter le partage et la consultation des analyses réalisées.

La génération automatique des rapports constitue une fonctionnalité essentielle du projet puisqu'elle permet aux développeurs d'obtenir une vue synthétique et exploitable de la qualité architecturale de leurs applications.

III.2.4 Détecteurs implémentés

La première version de GoReview AI intègre plusieurs détecteurs spécialisés permettant d'identifier automatiquement certains problèmes architecturaux récurrents.

God Struct / God Class Détecte les structures ou classes possédant trop de responsabilités, généralement caractérisées par un nombre élevé d'attributs et de méthodes.

High Complexity Calcule la complexité cyclomatique des fonctions afin d'identifier les portions de code difficiles à maintenir ou tester.

Concrete Dependency Détecte les dépendances directes vers des implémentations concrètes plutôt que vers des abstractions ou interfaces.

Détecteurs avancés basés LLM Le mode *--deep* permet également de détecter des problèmes plus complexes tels que :

- Envie de fonctionnalité,

- Violations de couche,
- Couplage temporel,
- Abstractions manquantes,
- Modèle de domaine anémique.

III.2.5 Autre applications et services

Tableau 6 - Applications et services

	Git est un système de gestion de versions qui facilite la collaboration et le suivi des modifications grâce à ses branches et outils de fusion puissants. [6] Son utilisation nous a permis de gérer efficacement les différentes versions du projet, de suivre l'évolution du code source et de travailler de manière structurée durant le développement.
	GitHub est une plateforme en ligne de Git qui nous a permis de centraliser notre code et d'en assurer une sauvegarde sécurisée. [7] Cette plateforme a permis de centraliser le code source, de sauvegarder le projet et de faciliter le suivi des différentes fonctionnalités développées.
	GanttProject est une application de gestion de projet gratuite pour les ordinateurs de bureau. Cet outil a été utilisé afin d'organiser les différentes phases du projet et de planifier les tâches de développement de manière structurée.

III.3 Implémentation

Après avoir présenté les outils, langages et bibliothèques utilisés, cette partie est consacrée à la mise en œuvre pratique du projet. Nous y exposons le résultat de notre travail sous forme de captures d'écran de l'application, afin d'illustrer le fonctionnement des différentes fonctionnalités développées.

L'application a été conçue autour d'un enchaînement clair et structuré : une interface en ligne de commande permettant de lancer une analyse statique instantanée ou enrichie par un modèle de langage ; un système de détection des problèmes architecturaux

produisant des findings classés par sévérité ; un module d'enrichissement LLM générant des explications et des exemples de refactorisation avant/après ; et enfin un système de scoring calculant un score de maintenabilité global accompagné d'un rapport exportable dans plusieurs formats.

III.3.1 Menu d'aide et options CLI

Le menu d'aide constitue le point d'entrée naturel de l'outil. La commande `./main -h` affiche l'ensemble des flags disponibles : le flag `--llm` pour activer l'enrichissement par intelligence artificielle, `--deep` pour une analyse approfondie par squelettes de packages, `-f` pour choisir le format de sortie (terminal, JSON ou HTML), `--min-score` pour intégrer l'outil dans un pipeline CI/CD avec un seuil de qualité configurable, ainsi que les options de configuration du fournisseur LLM (`--llm-provider`, `--llm-model`, `--llm-api-key`). Cette interface a été conçue pour offrir une prise en main rapide tout en exposant un paramétrage avancé pour les utilisateurs expérimentés.

```

$ ./main -h
GoReview AI analyses source code for architectural smells and suggests
design patterns to address them. Supports Go and Python. Run without --llm for
instant, fully local analysis.

Usage:
  gorview [dir] [flags]

Flags:
  --deep                deep LLM analysis - sends stripped package skeletons, one call per package
  --deep-timeout int    per-package LLM timeout for deep analysis in seconds (local models need more time) (default 300)
  -f, --format string    output format: terminal, json, html (default "terminal")
  -h, --help            help for gorview
  --llm                enable LLM enrichment (adds explanations and refactoring examples)
  --llm-api-key string   OpenAI API key (or set OPENAI_API_KEY)
  --llm-base-url string  Ollama base URL (default: http://localhost:11434)
  --llm-model string     LLM model name (default "qwen2.5-coder:7b")
  --llm-provider string  LLM provider: ollama, openai (default "ollama")
  --min-score int        exit 1 if maintainability score is below this threshold (CI mode)
  -o, --output string    write report to file instead of stdout
  --timeout int          static analysis + enrichment timeout in seconds (default 600)

```

Figure 21 – Menu d’aide (`./main -h`)

III.3.2 Analyse statique et enrichissement LLM

Les figures ci-dessous illustrent les deux modes d'analyse principaux de GoReview AI. En mode analyse statique (`./main -f .`), l'outil parcourt l'ensemble du code source et produit une liste de findings classés par sévérité — [CRITIQUE], [MOYEN] et [FAIBLE] — en indiquant pour chaque problème le fichier et la ligne concernés, la métrique associée (complexité cyclomatique, nombre de champs), ainsi que le patron de conception suggéré et l'effort de refactorisation estimé.

En mode enrichissement LLM (`./main --llm .`), chaque finding est transmis au modèle de langage configuré, qui génère une explication en langage naturel ainsi qu'un exemple concret de refactorisation avant/après en Go. Cette approche permet de passer

d'un simple constat technique à une recommandation architecturale directement exploitable par le développeur.

```

$ ./main -f .
[CRITIQUE] high_complexity /home/kali/pi/GoReview-AI/detectors/godstruct/detector.go:21
           Detector.Detect · CC=21
           → Patron suggéré : Stratégie · Effort : Élevé

[CRITIQUE] high_complexity /home/kali/pi/GoReview-AI/main.go:73
           run · CC=23
           → Patron suggéré : Stratégie · Effort : Élevé

[MOYEN] high_complexity /home/kali/pi/GoReview-AI/languges/golang/summarize.go:13
          Summarize · CC=19
          → Patron suggéré : Stratégie · Effort : Moyen

[MOYEN] high_complexity /home/kali/pi/GoReview-AI/languges/python/parser.go:51
          ParseFile · CC=19
          → Patron suggéré : Stratégie · Effort : Moyen

[MOYEN] god_struct /home/kali/pi/GoReview-AI/output/json.go:10
        jsonFinding · 11 champs · 0 méthodes
        → Patron suggéré : Façade · Effort : Moyen

[MOYEN] god_struct /home/kali/pi/GoReview-AI/core/finding.go:19
        Finding · 11 champs · 0 méthodes
        → Patron suggéré : Façade · Effort : Moyen

[FAIBLE] high_complexity /home/kali/pi/GoReview-AI/languges/golang/summarize.go:113
          computeCC · CC=11
          → Patron suggéré : Stratégie · Effort : Faible

[FAIBLE] high_complexity /home/kali/pi/GoReview-AI/detectors/complexity/detector.go:46
          cyclomaticComplexity · CC=11
          → Patron suggéré : Stratégie · Effort : Faible

[FAIBLE] high_complexity /home/kali/pi/GoReview-AI/languges/python/strip.go:19
          stripPython · CC=14
          → Patron suggéré : Stratégie · Effort : Faible

[FAIBLE] high_complexity /home/kali/pi/GoReview-AI/llm/deep_analyser.go:37
          ParseDeepFindings · CC=11

```

Figure 22 - Résultats de l'analyse statique (./main -f .)

```

[CRITIQUE] high_complexity /home/kali/pi/GoReview-AI/detectors/godstruct/detector.go:21
           Detector.Detect · CC=21
           + Patron suggéré : Stratégie · Effort : Moyen

The method Detector.Detect has a high cyclomatic complexity of 21, indicating it is overly complex and difficult to understand.

Before:
func (d *Detector) Detect(input string) (string, error) { ... }

After:
type DetectionStrategy interface {
    Detect(input string) (string, error)
}

func NewDefaultDetectionStrategy() DetectionStrategy {
    return &defaultDetectionStrategy{}
}

type defaultDetectionStrategy struct {}

func (s *defaultDetectionStrategy) Detect(input string) (string, error) { ... }

[CRITIQUE] high_complexity /home/kali/pi/GoReview-AI/main.go:73
           run · CC=23
           + Patron suggéré : Stratégie · Effort : Moyen

The 'run' component has a high cyclomatic complexity of 23, indicating complex control flow and multiple exit points. This makes the code difficult to understand and maintain.

Before:
func run() {
    // Complex logic with multiple conditional statements and nested loops
}

After:
type RunStrategy interface {
    Execute()
}

type SimpleRun struct {}

func (s *SimpleRun) Execute() {
    // Simplified logic for a single scenario
}

type ComplexRun struct {}

func (c *ComplexRun) Execute() {
    // Complex logic with multiple conditional statements and nested loops
}

```

Figure 23 - Enrichissement LLM avec exemples avant/après (./main --llm .)

III.3.3 Mode analyse approfondie et score final

Le mode `--deep` envoie au LLM un squelette strippé de chaque package — toutes les signatures et définitions de types sont conservées, mais les corps de fonctions sont remplacés par des commentaires résumant leur volume — offrant ainsi au modèle un contexte cross-fichiers qui lui permet de détecter des patterns architecturaux invisibles à l'analyse fichier par fichier.

À l'issue de l'analyse, quelle que soit la combinaison de flags utilisée, GoReview AI affiche un score de maintenabilité global sur 100, accompagné du décompte des findings par niveau de sévérité. Ce score synthétique constitue l'indicateur clé du projet et peut être utilisé comme porte de qualité dans un pipeline CI/CD via le flag `--min-score`.

```

L$ ./main --deep .
gorview: --deep requires --llm; enabling LLM automatically
[gorview] deep: analysing /home/kali/pi/GoReview-AI/core ...
[gorview] deep: no findings parsed from LLM response for /home/kali/pi/GoReview-AI/core
[gorview] deep: analysing /home/kali/pi/GoReview-AI/languages/golang ...
[gorview] deep: analysing /home/kali/pi/GoReview-AI/detectors/complexity ...
[gorview] deep: no findings parsed from LLM response for /home/kali/pi/GoReview-AI/detectors/complexity
[gorview] deep: analysing /home/kali/pi/GoReview-AI/detectors/concretedep ...
[gorview] deep: analysing /home/kali/pi/GoReview-AI/languages/python ...
[gorview] deep: analysing /home/kali/pi/GoReview-AI/detectors/python/complexity ...
[gorview] deep: no findings parsed from LLM response for /home/kali/pi/GoReview-AI/detectors/python/complexity
[gorview] deep: analysing /home/kali/pi/GoReview-AI/detectors/godstruct ...
[gorview] deep: no findings parsed from LLM response for /home/kali/pi/GoReview-AI/detectors/godstruct
[gorview] deep: analysing /home/kali/pi/GoReview-AI/llm ...
[gorview] deep: analysing /home/kali/pi/GoReview-AI/llm/openai ...
[gorview] deep: analysing /home/kali/pi/GoReview-AI/output ...
[gorview] deep: no findings parsed from LLM response for /home/kali/pi/GoReview-AI/output
[gorview] deep: analysing /home/kali/pi/GoReview-AI/detectors/python ...
[gorview] deep: no findings parsed from LLM response for /home/kali/pi/GoReview-AI/detectors/python
[gorview] deep: analysing /home/kali/pi/GoReview-AI ...
[gorview] deep: no findings parsed from LLM response for /home/kali/pi/GoReview-AI
[gorview] deep: analysing /home/kali/pi/GoReview-AI/detectors ...
[gorview] deep: no findings parsed from LLM response for /home/kali/pi/GoReview-AI/detectors
[gorview] deep: analysing /home/kali/pi/GoReview-AI/detectors/python/godclass ...
[gorview] deep: no findings parsed from LLM response for /home/kali/pi/GoReview-AI/detectors/python/godclass
[gorview] deep: analysing /home/kali/pi/GoReview-AI/llm/ollama ...

```

Figure 24 - Mode analyse approfondie (`./main --deep .`)

```

score de maintenabilité : 30 / 100
critique : 2   moyen : 4   faible : 4

```

Figure 25 - Score de maintenabilité final

III.3.4 Rapport HTML interactif

GoReview AI propose également une sortie au format HTML, générant un rapport auto-contenu et interactif accessible directement dans un navigateur. Ce rapport présente en haut de page le score de maintenabilité global sur 100, ainsi qu'un résumé visuel du nombre de findings répartis par niveau de sévérité — Critique, Moyen et Faible — permettant d'appréhender l'état architectural du projet en un coup d'œil.

Les findings sont ensuite listés par catégorie, chacun pouvant être déplié pour révéler le détail complet : les métriques associées (complexité cyclomatique, nombre de champs), une explication en langage naturel générée par le LLM, ainsi qu'un exemple de refactorisation avant/après présenté sous forme de blocs de code côte à côte avec un bouton de copie. Ce format est particulièrement adapté au partage en équipe ou à l'intégration dans une revue de projet, car il ne nécessite aucune dépendance externe pour être consulté.

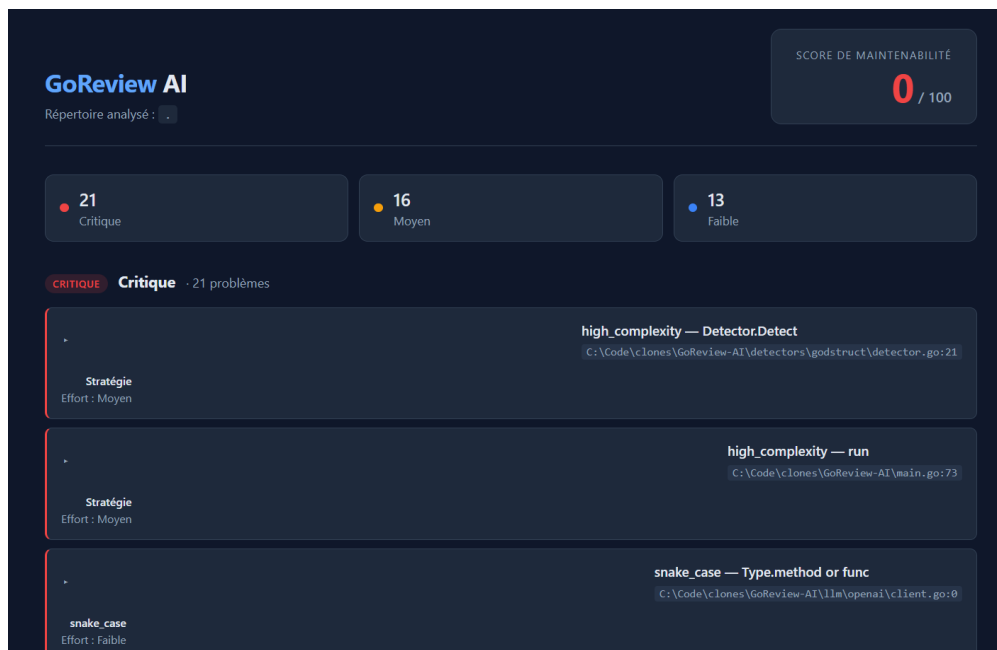


Figure 26 - Vue d'ensemble du rapport HTML (score et findings par sévérité)

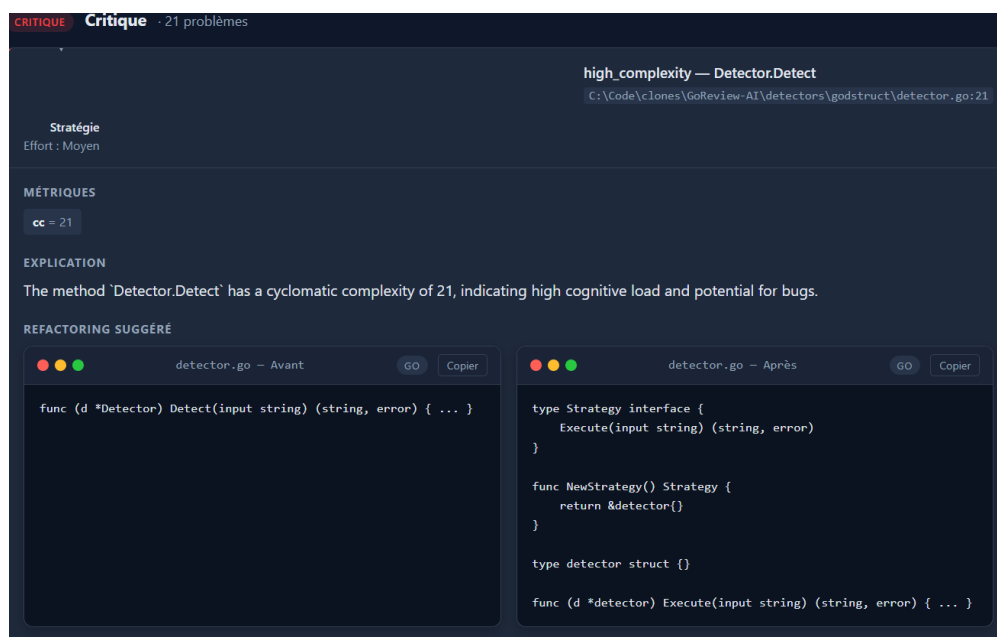


Figure 27 - Détail d'un finding avec explication LLM et refactorisation suggérée

III.4 Limites actuelles du projet

Malgré les fonctionnalités déjà implémentées, certaines limites subsistent dans la version actuelle de GoReview AI :

- Le nombre de détecteurs architecturaux reste encore limité,
- L'analyse Python repose partiellement sur des techniques textuelles moins précises qu'une analyse AST complète,
- Les performances des analyses profondes dépendent fortement des capacités matérielles lors de l'utilisation de modèles LLM locaux,
- Certains problèmes architecturaux complexes nécessitent encore une validation humaine.

Ces limites ouvrent plusieurs perspectives d'amélioration pour les futures versions du projet.

III.5 Conclusion

Ce chapitre a permis de présenter les différentes technologies, outils et fonctionnalités mis en œuvre dans le cadre du développement de GoReview AI. À travers l'utilisation de bibliothèques adaptées à l'analyse de code source, au traitement des données et à l'intégration de modèles d'intelligence artificielle, nous avons pu concrétiser les fonctionnalités définies lors des phases d'analyse et de conception. Les différentes interfaces, diagrammes et captures présentés permettent d'illustrer le fonctionnement global de l'application ainsi que les principales étapes du pipeline d'analyse architecturale. L'application développée offre ainsi une solution capable d'automatiser une partie importante de l'évaluation de la qualité logicielle tout en fournissant des recommandations exploitables pour améliorer la maintenabilité des projets.

Conclusion générale

Ce projet nous a permis de concevoir et de réaliser GoReview AI, une plateforme d'analyse architecturale capable de combiner des techniques d'analyse statique traditionnelles avec les capacités offertes par les modèles d'intelligence artificielle modernes. L'objectif principal était de fournir un outil capable d'assister les développeurs et les responsables techniques dans l'identification des problèmes de qualité logicielle, de maintenabilité et d'architecture au sein des projets logiciels.

En nous appuyant sur des technologies modernes telles que Go pour le développement principal de l'application, les parseurs d'analyse syntaxique, les modèles LLM ainsi que différents outils de génération de rapports et d'automatisation, nous avons pu développer une solution modulaire, flexible et évolutive. Le projet nous a également permis d'approfondir plusieurs notions importantes liées à l'ingénierie logicielle, notamment l'analyse statique, les architectures logicielles, l'intégration des systèmes d'intelligence artificielle et l'optimisation des pipelines d'analyse.

Le respect des bonnes pratiques de développement a constitué un aspect fondamental de notre démarche. L'utilisation de Git pour le versionnage, l'organisation modulaire du code ainsi que la mise en place d'une architecture claire ont contribué à améliorer la maintenabilité et la qualité globale du projet.

Au-delà des aspects techniques, ce travail nous a également permis de développer des compétences organisationnelles et collaboratives importantes. La gestion des tâches via Notion, le travail en équipe ainsi que la répartition progressive des responsabilités nous ont permis de mener le projet de manière structurée tout en restant flexibles face aux contraintes rencontrées durant le développement.

Bien que GoReview AI réponde aux principaux objectifs fixés au début du projet, plusieurs perspectives d'amélioration peuvent être envisagées pour les futures versions :

- Ajouter le support d'autres langages de programmation afin d'élargir les capacités d'analyse de la plateforme,
- Améliorer les détecteurs architecturaux afin d'identifier des problèmes plus complexes et plus spécifiques,

- Intégrer davantage de fonctionnalités destinées aux environnements CI/CD et DevOps,
- Optimiser davantage les mécanismes de réduction et de compression du contexte transmis aux modèles LLM,
- Développer une interface graphique complète afin de rendre l'outil plus accessible à des utilisateurs non techniques,
- Étendre les capacités de reporting et de visualisation des résultats.

En définitive, ce projet représente une expérience particulièrement enrichissante aussi bien sur le plan technique qu'organisationnel. Il constitue une base solide pour le développement futur d'outils intelligents d'analyse logicielle capables d'assister efficacement les développeurs dans l'amélioration continue de la qualité de leurs projets.

Bibliographie

- [1] « Visual Studio Code » [En ligne]. Disponible sur : <https://code.visualstudio.com/>.
- [2] « Go » [En ligne]. Disponible sur : [https://fr.wikipedia.org/wiki/Go_\(langage\)](https://fr.wikipedia.org/wiki/Go_(langage))
- [3] « Cobra » [En ligne]. Disponible sur : <https://cobra.dev/>
- [4] « Ollama » [En ligne]. Disponible sur : <https://ollama.com/>
- [5] « OpenAI » [En ligne]. Disponible sur : <https://openai.com/>
- [6] « Git » [En ligne]. Disponible sur : <https://git-scm.com/>
- [7] « Github » [En ligne]. Disponible sur : <https://fr.wikipedia.org/wiki/GitHub>